

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

Отчёт

по лабораторной работе №5

по курсу «Разработка кроссплатформенных приложений»

на тему «Сетевое взаимодействие в Java»

Вариант 9

Выполнили студенты группы 23ВВИ2:

Юров Д.М.

Лисов Е.А.

Кочегин В.В.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Карамышева Н.С.

Пенза 2026

Содержание

1. Модификация приложения-клиента.....	4
1.1 Класс Message	4
1.2 Класс ClientUdpService	5
1.3 Изменения в классе JFrameGUI.....	7
1.4 Описание работы клиента.....	8
2. Разработка серверной части приложения	9
2.1 Класс ServerMain	9
2.2 Класс User	10
2.3 Класс Register.....	10
2.4 Класс Message	11
2.5 Класс CalcResult	11
2.6 Класс Sender.....	12
2.7 Класс Recipient.....	12
2.8 Описание работы сервера	12
3 Тестирование и отладка программы	14
3.1 Первый тест – регистрация пользователей	14
3.2 Второй тест – Вычисление интеграла с одного клиента.....	16
3.3 Тест третий – Вычисление интеграла с нескольких клиентов	18
3.4 Тест четвертый – Вычисление интеграла после закрытия запущенных клиентов	21
Приложение 1 – исходный код приложения-клиента	26
Приложение 1.1 – содержимое файла «Prog6.java».....	26
Приложение 1.2 – содержимое файла «Integration.java».....	27
Приложение 1.3 – содержимое файла «JFrameGUI.java»	29
Приложение 1.4 – содержимое файла «OutOfLimitException.java»	50
Приложение 1.5 – содержимое файла «Message.java»	51
Приложение 1.6 – содержимое файла «ClientUdpService.java».....	54
Приложение 2 – исходный код приложения-сервера.....	58
Приложение 2.1 – содержимое файла «CalcRequest.java»	58
Приложение 2.2 – содержимое файла «Message.java»	61
Приложение 2.3 – содержимое файла «Recipient.java»	64
Приложение 2.4 – содержимое файла «Register.java».....	65
Приложение 2.5 – содержимое файла «Sender.java»	67
Приложение 2.6 – содержимое файла «ServerMain.java»	68
Приложение 2.7 – содержимое файла «User.java».....	75

Цель работы: научиться создавать клиент-серверные приложения с использованием стандартных классов Java.

Задание: модифицировать приложение из предыдущей лабораторной работы, реализовав клиент-серверную архитектуру, обеспечивающую распределенное вычисление определенного интеграла на нескольких вычислительных узлах (клиентах) при этом каждый узел использует несколько нитей, как в предыдущей работе. Сервер не занимается вычислениями, а лишь реализует взаимодействие с пользователем и агрегацию результатов вычислений от клиентов. Нечетные варианты используют протокол UDP, а четные TCP.

Ход работы:

1. Модификация приложения-клиента

В лабораторной работе номер 5 клиентское приложение выполняло вычисление локально при нажатии кнопки «Вычислить», причем вычисления происходили в 9 одновременных одинаковых потоках. Приложение, разработанное в ходе выполнения 5 лабораторной работы было модифицировано – теперь клиент не выполняет полное вычисление самостоятельно, а взаимодействует с сервером по протоколу UDP). Роль клиента теперь разбита на 2 составляющие: 1 – графический клиент – отправляет запрос на вычисление серверу; 2 – вычислительный узел – получает свою часть задачи на вычисление от сервера и считает её в 9 одинаковых потоках.

1.1 Класс Message

Был добавлен класс Message, который отвечает за хранение, парсинг и генерацию UDP-сообщений. Класс нужен для удобного взаимодействия с сообщениями. Все сообщения между клиентом и сервером имеют единый вид: «msgType;requestId;partNum;a;b;h;res», где:

- msgType – тип сообщения;
- requestId – номер запроса на вычисление;
- partNum – номер части вычисления;
- a – нижняя граница интегрирования;
- b – верхняя граница интегрирования;
- h – шаг интегрирования;
- res – результат вычисления.

В классе Message объявлены следующие поля:

- private final String rawText хранит исходное сообщение целиком;
- private final String msgType хранит тип сообщения;
- private final int requestId хранит идентификатор вычислительного запроса;
- private final int partNum хранит номер части задачи;

- `private final double a, b, h` хранят параметры интегрирования;
- `private final double res` хранит результат вычисления.

Конструктор класса принимает строку UDP-сообщения и разбивает её на части, если сообщение имеет неверный формат, то выбрасывается исключение. Были реализованы методы-«геттеры», которые возвращают значения каждой хранимой в классе переменной. Были реализованы методы проверки типа сообщений для удобных проверок извне класса.

Были реализованы методы создания сообщений – они создают сообщения по заданным параметрам:

- метод `createRegister` создаёт сообщение регистрации клиента (`REGISTER;0;0;0;0;0;0`);
- метод `createCalcRequest` создаёт сообщение запроса на вычисление (`CALC_REQUEST;requestId;0;a;b;h;0`);
- метод `createCalcResult` создает сообщение с результатом вычисления полученной части интеграла (`CALC_RESULT;requestId;partNum;0;0;0;result`);
- метод `createStatusOk` создает сообщение подтверждения активного состояния клиента (`STATUS_OK;requestId;0;0;0;0;0`);

1.2 Класс `ClientUdpService`

Класс `ClientUdpService` отвечает за всю сетевую работу клиента – отвечает за отправку и получение UDP-сообщений.

В классе объявлены следующие поля:

- `private final JFrameGUI gui` хранит ссылку на графическое окно клиента. Она нужна, чтобы передавать результат вычисления обратно в интерфейс;
- `private final InetAddress serverAddress` хранит IP-адрес сервера;
- `private final int serverPort` хранит UDP-порт сервера;
- `private DatagramSocket socket` используется для отправки и получения UDP-пакетов;

- `private volatile boolean running` управляет работой фонового потока приёма сообщений.

Конструктор класса получает ссылку на GUI, адрес сервера и порт сервера. внутри конструктора определяется IP-адрес и порт сервера, создается UDP-сокеты клиента, клиенту присуждается свободный порт из заданного диапазона.

В классе реализовано несколько методов:

- метод `createSocketInRange` создаёт сокет на ближайшем свободном порту в заданном диапазоне. Нужен для автоматического выбора свободного UDP-порта клиентом;
- метод `getLocalPort` возвращает локальный UDP-порт, который выбрал клиент;
- метод `start` запускает фоновый поток, который постоянно слушает входящие UDP-сообщения от сервера;
- метод `stop` останавливает работу UDP-клиента и закрывает сокет – нужен для корректного завершения работы приложения;
- метод `registerOnServer` отправляет запрос регистрации клиента на сервере - отправляет сообщение «REGISTER;0;0;0;0;0;0»;
- метод `sendCalcRequest` отправляет серверу запрос на вычисление – вызывается после нажатия на кнопку «Вычислить», клиент передаёт серверу номер запроса, нижнюю границу интегрирования, верхнюю границу интегрирования и шаг интегрирования;
- метод `sendCalcResult` отправляет серверу результат вычисления полученной ранее части интеграла;
- метод `sendStatusOk` отправляет сообщение («STATUS_OK») серверу – нужен для подтверждения активного состояния клиента перед распределением задач вычисления между клиентами;
- метод `sendToServer` отправляет UDP-сообщение серверу в виде массива байтов в `DatagramPacket` – метод используется всеми остальными методами отправки;
- метод `listenLoop` работает в отдельном фоновом потоке – постоянно ожидает UDP-пакеты от сервера, когда сообщение получено, оно передается в метод `handleMessage`;

- метод `handleMessage` определяет тип полученного сообщения и выполняет требуемое сообщением действие.

1.3 Изменения в классе `JFrameGUI`

В классе `JFrameGUI` была изменена работа кнопки «Вычислить» и добавлена работа с UDP-клиентом.

В класс были добавлены новые поля:

- `udpService` хранит объект, отвечающий за UDP-обмен с сервером;
- `requestCounter` нужен для генерации уникальных номеров запросов;
- `requestRows` хранит связь между `requestId` и строкой таблицы. Это нужно, чтобы при получении `FINAL_RESULT` клиент знал, в какую строку таблицы записать результат;
- `SERVER_HOST` хранит IP-адрес сервера;
- `SERVER_PORT` хранит UDP-порт сервера.

В классе были определены новые методы:

- метод `initUdpClient` создает объект `ClientUdpService`, запускает фоновый поток приёма сообщений и отправляет серверу запрос на регистрацию. Метод вызывается при создании окна `JFrameGUI`;
- был изменен метод `jButtonCalcActionPerformed` – теперь он проверяет, выбрана ли строка таблицы, считывает переменные из выбранной строки, создает новый `requestId`, записывает в таблицу текст «ожидание...», отправляет серверу запрос на вычисление;
- метод `recieveFinalResult` вызывается, когда клиент получает от сервера сообщение типа «`FINAL_RESULT`». Метод находит строку таблицы по `requestId`, записывает итоговый результат в колонку «Результат», обновляет объект в коллекции `entries`, удаляет `requestId` из `requestRows`;
- метод `calculateServerTask` вызывается при получении от сервера запроса на вычисление – он запускает отдельный поток, чтобы не блокировать приём UDP-сообщений. После вычисления отправляет серверу сообщение с результатом вычисления;

- Метод `calculatePartInThreads` выполняет вычисление полученной от сервера части интеграла в 9 потоков, каждый из которых вычисляет полученную от сервера часть интеграла;
- метод `showNetworkStatus` выводит служебную информацию о состоянии сети.

1.4 Описание работы клиента

Клиентское приложение запускается как обычное Swing-приложение.

При запуске клиент создает графическое окно, создает UDP-сокеты, выбирает свободный локальный порт из заданного диапазона, запускает фоновый поток приёма сообщений, отправляет серверу сообщение типа REGISTER.

После регистрации клиент ждёт сообщений от сервера.

Если клиент получает сообщение типа STATUS_CHECK, он отправляет серверу сообщение типа STATUS_OK.

Если клиент получает CALC_TASK, то он вычисляет полученную часть интеграла в 9 потоках и отправляет серверу сообщение типа CALC_RESULT.

Если клиент получает сообщение типа FINAL_RESULT, он записывает итоговый результат в таблицу.

Если пользователь после ввода и добавления значений в таблицу нажимает кнопку «Вычислить», клиент создаёт сообщение типа CALC_REQUEST и отправляет его серверу. Самостоятельно интеграл клиент не считает.

2. Разработка серверной части приложения

Серверное приложение реализовано как отдельное консольное приложение – так как Java является кроссплатформенным языком программирования – серверное приложение будет запущено на сервере Linux Ubuntu Server версии 24.04.4 LTS.

Сервер не имеет графического интерфейса – только консольный. Он выполняет роль координатора распределенных вычислений.

2.1 Класс ServerMain

Класс ServerMain является главным классом серверного приложения.

В классе объявлены поля:

- `private static final int SERVER_PORT` задаёт UDP-порт сервера;
- `private static final int STATUS_CHECK_TIMEOUT_MS` задаёт время ожидания ответа от клиентов при проверке активности;
- `private static final HashMap<Integer, CalcRequest> requests` хранит активные вычислительные запросы по их `requestId`.

В классе объявлены следующие методы:

- метод `main` создает `DatagramSocket` на порту 5000 и запускает бесконечный цикл ожидания получения сообщения. При получении сообщения метод получает текст сообщения, создает объект `Message`, определяет тип сообщения и вызывает соответствующий обработчик;
- метод `handleRegister` обрабатывает регистрацию клиента, после чего отправляет клиенту сообщение об успешной регистрации;
- метод `handleCalcRequest` обрабатывает запрос на вычисление от клиента – определяет главного клиента по `Ip` и порту отправителя, берет список зарегистрированных клиентов, проверяет, какие клиенты сейчас активны, создает объект `CalcRequest`, сохраняет его в `HashMap requests`, запускает распределение задачи между активными клиентами;
- метод `checkActiveUsers` проверяет, какие зарегистрированные клиенты активны в данный момент – отправляет всем зарегистрированным

клиентам сообщение типа STATUS_CHECK, ждёт ответы типа STATUS_OK, формирует список activeUsers, возвращает список из клиентов, которые ответили на STATUS_CHECK;

- метод distributeTask обрабатывает частичный результат от клиента – находит объект CalcRequest по requestId, добавляет полученный результат к полной сумме, проверяет завершено ли вычисление, если вычисление завершено, то отправляет главному клиенту конечный результат.

2.2 Класс User

Класс User хранит данные о зарегистрированном клиенте.

Были созданы следующие поля класса:

- private final InetAddress clientIp хранит IP-адрес клиента;
- private final int clientPort хранит UDP-порт клиента, который сервер увидел при получении пакета;
- private final String clientName хранит внутреннее имя клиента, например Client-1.

Были созданы методы-«геттеры», возвращающие значения полей класса.

2.3 Класс Register

Класс Register отвечает за регистрацию клиентов и хранение их списка.

Было создано поле класса private static final ArrayList<User> users, хранящие список всех зарегистрированных клиентов.

Были созданы следующие методы класса:

- метод registration занимается регистрацией клиентов – он получает IP и порт клиента из DatagramPacket, проверяет, не зарегистрирован ли уже этот клиент, создаёт объект User, добавляет его в список users;
- метод findByAdress ищет клиента в списке по его IP и порту;

- метод `getUsersCopy` возвращает копию списка зарегистрированных клиентов;
- метод `removeUser` удаляет клиента из списка;
- метод `remove users` удаляет сразу несколько клиентов;
- метод `printUsers` выводит список зарегистрированных клиентов в консоль сервера.

2.4 Класс Message

Класс `Message` является серверным вариантом такого же класса для клиента – хранит, разбирает и создает UDP-сообщения.

Были реализованы следующие методы:

- метод `createRegisterOk` создает сообщение подтверждения регистрации;
- метод `createError` создает сообщение об ошибке;
- метод `createStatusCheck` создает сообщение проверки активности клиента;
- метод `createCalcTask` создает задачу для вычислительного клиента;
- метод `createFinalResult` создает сообщение с итоговым результатом вычисления.

2.5 Класс CalcResult

Класс `CalcResult` хранит данные об одном вычислительном запросе.

Были реализованы следующие поля класса:

- `private final int requestId` хранит номер вычисления;
- `private final User mainUser` хранит клиента, который нажал «Вычислить»;
- `private final ArrayList<User> calcUsers` хранит список клиентов, между которыми распределена задача;
- `private final double a, b, h` хранят параметры интегрирования;
- `private final int expectedResults` хранит количество ожидаемых частичных результатов;

- `private int receivedResults` хранит количество уже полученных результатов;
- `private double totalResult` хранит сумму частичных результатов.

Были реализованы следующие методы класса:

- метод `addResult` добавляет частичный результат к общей сумме и увеличивает счетчик полученных результатов;
- метод `isCompleted` проверяет, получены ли все ожидаемые результаты.

2.6 Класс Sender

Класс `Sender` отвечает только за отправку UDP-сообщений.

Метод `send` преобразует строку сообщения в массив байтов, создает `DatagramPacket`, отправляет пакет на IP и порт указанного пользователя.

2.7 Класс Recipient

Класс `Recipient` отвечает только за получение UDP-сообщений.

Метод `receive` создает буфер, ожидающий входящий UDP-пакет и возвращает `DatagramPacket`.

Метод `getText` преобразует данные из UDP-пакета в строку.

2.8 Описание работы сервера

Сервер запускается на сервере с ОС Linux Ubuntu Server версии 24.04.4 LTS как консольное Java приложение. После запуска сервер слушает UDP-порт 5000. Когда сервер получает сообщение типа `REGISTER`, он добавляет клиента в список зарегистрированных клиентов.

Когда сервер получает сообщение типа `CALC_REQUEST` он определяет главного клиента, берет список зарегистрированных клиентов, проверяет статус клиентов, формирует список вычислений для активных клиентов, делит интервал

интегрирования между активными клиентами и отправляет каждому из них сообщение типа CALC_TASK.

Когда сервер получает сообщение типа CALC_RESULT, он добавляет частичный результат к общей сумме.

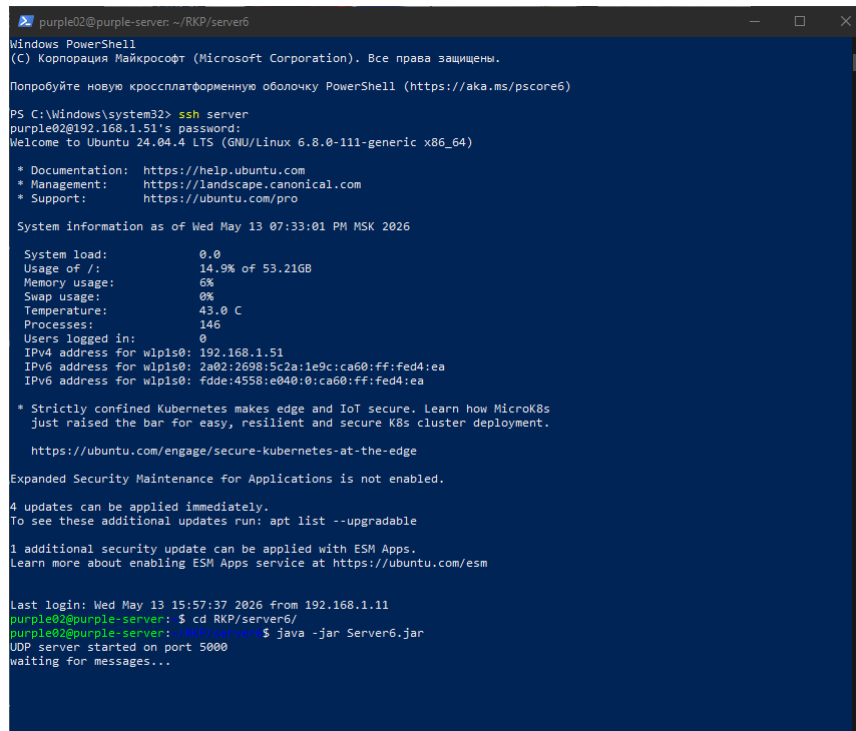
Когда сервер получает все частичные результаты, он отправляет главному клиенту сообщение типа FINAL_RESULT.

3 Тестирование и отладка программы

3.1 Первый тест – регистрация пользователей

В данном тесте будет проверка регистрации клиентов сервером.

На сервере с ОС Linux Ubuntu Server было запущено серверное приложение (рис. 1).



```
purple02@purple-server: ~/RKP/server6
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.
Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/pscore6)
PS C:\Windows\system32> ssh server
purple02@192.168.1.51's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed May 13 07:33:01 PM MSK 2026

System load:          0.0
Usage of /:            14.9% of 53.21GB
Memory usage:         6%
Swap usage:           0%
Temperature:          43.0 C
Processes:             146
Users logged in:       0
IPv4 address for wlp1s0: 192.168.1.51
IPv6 address for wlp1s0: 2a02:2698:5c2a:1e9c:ca60:ff:fed4:ea
IPv6 address for wlp1s0: fdde:4558:e040:0:ca60:ff:fed4:ea

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.
   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance For Applications is not enabled.

4 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Wed May 13 15:57:37 2026 from 192.168.1.11
purple02@purple-server: $ cd RKP/server6/
purple02@purple-server: ~/RKP/server6 $ java -jar Server6.jar
UDP server started on port 5000
waiting for messages...
```

Рисунок 1 - запуск серверного приложения

После запуска серверного приложения было запущен один клиент (рис. 2).

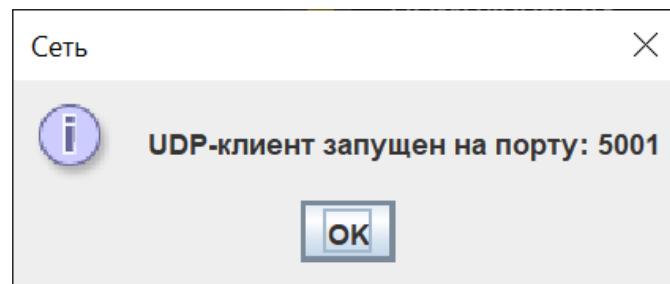


Рисунок 2 - запуск первого клиента

В консольном приложении на сервере появилось сообщение о регистрации пользователя (рис. 3).

```
received from 192.168.1.11:5001: REGISTER;0;0;0;0;0
client registered: Client-1 192.168.1.11:5001

sent to Client-1 192.168.1.11:5001: REGISTER_OK;0;0;0;0;0
=== registered users ===
Client-1 192.168.1.11:5001
```

Рисунок 3 - успешная регистрация первого клиента

Было запущено второе приложение (рис. 4).

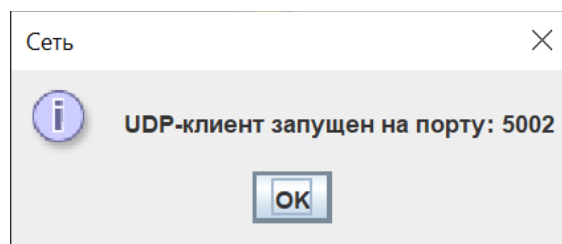


Рисунок 4 - запуск второго клиента

Регистрация на сервере успешна (рис. 5).

```
received from 192.168.1.11:5002: REGISTER;0;0;0;0;0
client registered: Client-2 192.168.1.11:5002

sent to Client-2 192.168.1.11:5002: REGISTER_OK;0;0;0;0;0
=== registered users ===
Client-1 192.168.1.11:5001
Client-2 192.168.1.11:5002
```

Рисунок 5 - успешная регистрация второго клиента

Было запущено третье приложение (рис. 6).

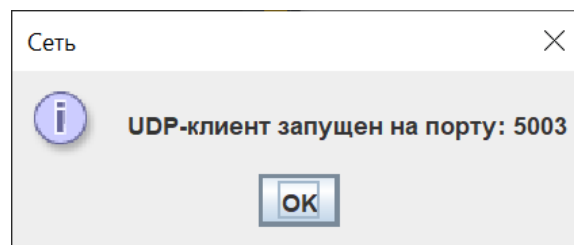


Рисунок 6 - запуск третьего клиента

Регистрация на сервере успешна (рис. 7).

```
received from 192.168.1.11:5003: REGISTER;0;0;0;0;0
client registered: Client-3 192.168.1.11:5003

sent to Client-3 192.168.1.11:5003: REGISTER_OK;0;0;0;0;0
=== registered users ===
Client-1 192.168.1.11:5001
Client-2 192.168.1.11:5002
Client-3 192.168.1.11:5003
```

Рисунок 7 - успешная регистрация третьего клиента

Было запущено четвертое приложение (рис. 8)

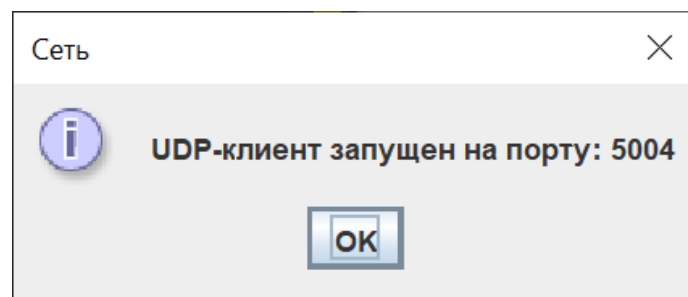


Рисунок 8 - запуск четвертого клиента

Регистрация на сервере успешна (рис. 9).

```
received from 192.168.1.11:5004: REGISTER;0;0;0;0;0
client registered: Client-4 192.168.1.11:5004

sent to Client-4 192.168.1.11:5004: REGISTER_OK;0;0;0;0;0
=== registered users ===
Client-1 192.168.1.11:5001
Client-2 192.168.1.11:5002
Client-3 192.168.1.11:5003
Client-4 192.168.1.11:5004
```

Рисунок 9 - успешная регистрация четвертого клиента

3.2 Второй тест – Вычисление интеграла с одного клиента

На одном из запущенных клиентов была добавлена строка со случайными значениями интегрирования (рис. 10), после чего была нажата кнопка «Вычислить».

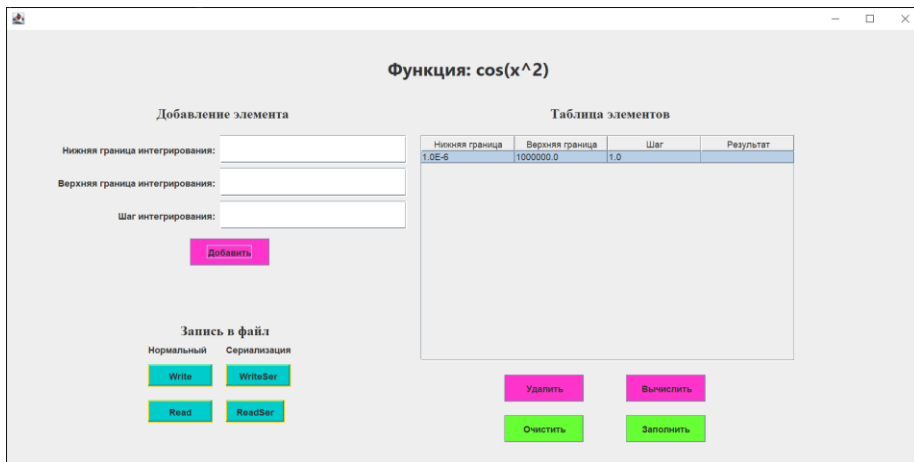


Рисунок 10 - добавление случайных значений 1

После чего на сервер пришел запрос на вычисление, после обработки которого был проверен список активных клиентов, задача вычисления была разбита на 4 части и отправлена на все активные клиенты. От каждого из клиентов был получен результат вычисления своей части интеграла, после чего сервер отправил итоговый результат главному клиенту (рис. 11).

```

purple02@purple-server: ~/RKP/server6
received from 192.168.1.11:5004: CALC_REQUEST;1001;0;1.0E-6;1000000.0;1.0;0
checking active clients before calculation...
sent to Client-1 192.168.1.11:5001: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-2 192.168.1.11:5002: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-3 192.168.1.11:5003: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-4 192.168.1.11:5004: STATUS_CHECK;1001;0;0;0;0;0
status response received from 192.168.1.11:5001: STATUS_OK;1001;0;0;0;0;0
client is active: Client-1 192.168.1.11:5001
status response received from 192.168.1.11:5002: STATUS_OK;1001;0;0;0;0;0
client is active: Client-2 192.168.1.11:5002
status response received from 192.168.1.11:5004: STATUS_OK;1001;0;0;0;0;0
client is active: Client-4 192.168.1.11:5004
status response received from 192.168.1.11:5003: STATUS_OK;1001;0;0;0;0;0
client is active: Client-3 192.168.1.11:5003
active clients: 4/4

new calc request: requestId=1001, mainUser=Client-4 192.168.1.11:5004, calcUsers=4, a=1.0E-6, b=1000000.0, h=1.0, received=0/4, totalResult=0.0
distributing task between 4 active clients...

sent to Client-1 192.168.1.11:5001: CALC_TASK;1001;0;1.0E-6;250000.000000075;1.0;0
sent to Client-2 192.168.1.11:5002: CALC_TASK;1001;1;250000.000000075;500000.00000005;1.0;0
sent to Client-4 192.168.1.11:5004: CALC_TASK;1001;2;500000.00000005;750000.0000002501;1.0;0
sent to Client-3 192.168.1.11:5003: CALC_TASK;1001;3;750000.0000002501;1000000.0;1.0;0

received from 192.168.1.11:5004: CALC_RESULT;1001;2;0;0;0;420.41905572093356
received result for request 1001, part 2: 420.41905572093356
progress: 1/4

received from 192.168.1.11:5002: CALC_RESULT;1001;1;0;0;0;-269.4097382826784
received result for request 1001, part 1: -269.4097382826784
progress: 2/4

received from 192.168.1.11:5001: CALC_RESULT;1001;0;0;0;0;-122.92575041529751
received result for request 1001, part 0: -122.92575041529751
progress: 3/4

received from 192.168.1.11:5003: CALC_RESULT;1001;3;0;0;0;53.61363644838814
received result for request 1001, part 3: 53.61363644838814
progress: 4/4

sent to Client-4 192.168.1.11:5004: FINAL_RESULT;1001;0;0;0;0;81.69720347134577
request completed: 1001
final result: 81.69720347134577
sent final result to: Client-4 192.168.1.11:5004

```

Рисунок 11 - вывод консольного приложения во время выполнения распределения задачи

3.3 Тест третий – Вычисление интеграла с нескольких клиентов

В предыдущих тестах было запущено 4 клиента, на каждом из них добавил по одной строке для вычисления (рис. 12).

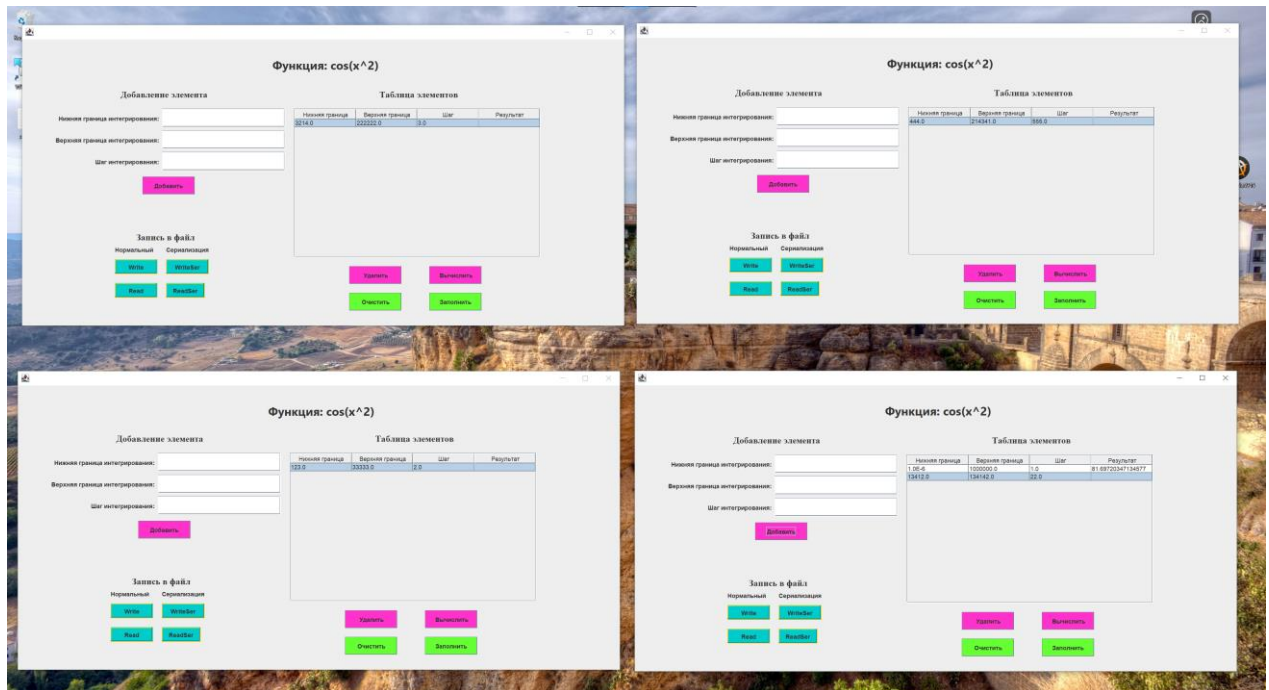


Рисунок 12 - запущенные клиенты

Во втором клиенте запустил вычисление строки со случайными значениями (рис. 13).

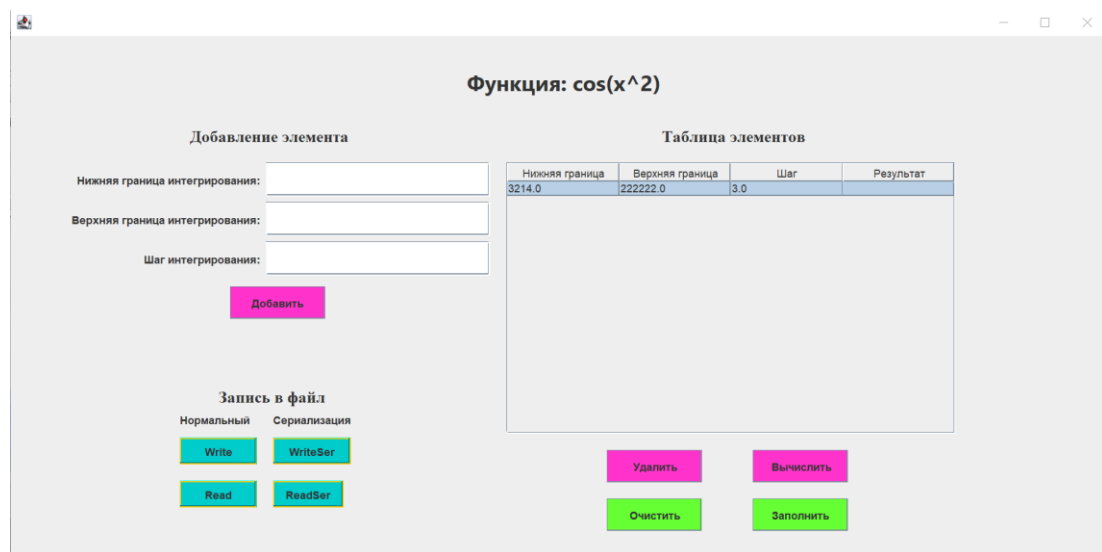


Рисунок 13 - запуск вычисления на втором клиенте

Сервер принял запрос вычисления от второго клиента и обработал его (рис. 14).

```

purple02@purple-server: ~/RKP/server6
received from 192.168.1.11:5002: CALC_REQUEST;1001;0;3214.0;222222.0;3.0;0
checking active clients before calculation...

sent to Client-1 192.168.1.11:5001: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-2 192.168.1.11:5002: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-3 192.168.1.11:5003: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-4 192.168.1.11:5004: STATUS_CHECK;1001;0;0;0;0;0
status response received from 192.168.1.11:5002: STATUS_OK;1001;0;0;0;0;0
client is active: Client-2 192.168.1.11:5002
status response received from 192.168.1.11:5001: STATUS_OK;1001;0;0;0;0;0
client is active: Client-1 192.168.1.11:5001
status response received from 192.168.1.11:5004: STATUS_OK;1001;0;0;0;0;0
client is active: Client-4 192.168.1.11:5004
status response received from 192.168.1.11:5003: STATUS_OK;1001;0;0;0;0;0
client is active: Client-3 192.168.1.11:5003
active clients: 4/4

new calc request: requestId=1001, mainUser=Client-2 192.168.1.11:5002, calcUsers=4, a=3214.0, b=222222.0, h=3.0, receive
d=0/4, totalResult=0.0
distributing task between 4 active clients...

sent to Client-2 192.168.1.11:5002: CALC_TASK;1001;0;3214.0;57966.0;3.0;0
sent to Client-1 192.168.1.11:5001: CALC_TASK;1001;1;57966.0;112718.0;3.0;0
sent to Client-4 192.168.1.11:5004: CALC_TASK;1001;2;112718.0;167470.0;3.0;0
sent to Client-3 192.168.1.11:5003: CALC_TASK;1001;3;167470.0;222222.0;3.0;0

received from 192.168.1.11:5002: CALC_RESULT;1001;0;0;0;0;-555.8491088351159
received result for request 1001, part 0: -555.8491088351159
progress: 1/4

received from 192.168.1.11:5001: CALC_RESULT;1001;1;0;0;0;-160.92565421057657
received result for request 1001, part 1: -160.92565421057657
progress: 2/4

received from 192.168.1.11:5003: CALC_RESULT;1001;3;0;0;0;-261.19072362173245
received result for request 1001, part 3: -261.19072362173245
progress: 3/4

received from 192.168.1.11:5004: CALC_RESULT;1001;2;0;0;0;239.01634223840688
received result for request 1001, part 2: 239.01634223840688
progress: 4/4

sent to Client-2 192.168.1.11:5002: FINAL_RESULT;1001;0;0;0;0;-738.949144429018
request completed: 1001
final result: -738.949144429018
sent final result to: Client-2 192.168.1.11:5002

```

Рисунок 14 - обработка запроса от второго клиента

После получения результата от сервера в клиенте обновилось поле результата (рис. 15).

Функция: $\cos(x^2)$

Добавление элемента

Нижняя граница интегрирования:

Верхняя граница интегрирования:

Шаг интегрирования:

Добавить

Запись в файл

Нормальный

Серялизация

Write

WriteSer

Read

ReadSer

Удалить

Вычислить

Очистить

Заполнить

Таблица элементов

Нижняя граница	Верхняя граница	Шаг	Результат
3214.0	222222.0	3.0	-738.949144429018

Рисунок 15 - результат для второго клиента получен

В третьем клиенте запустил вычисление строки (рис. 16).

Функция: $\cos(x^2)$

Добавление элемента

Нижняя граница интегрирования:

Верхняя граница интегрирования:

Шаг интегрирования:

Добавить

Запись в файл

Нормальный Сериализация

Write WriteSer

Read ReadSer

Удалить Вычислить

Очистить Заполнить

Нижняя граница	Верхняя граница	Шаг	Результат
444.0	214341.0	555.0	

Рисунок 16 - запуск вычисления на первом клиенте

Сервер принял запрос вычисления от первого клиента и обработал его (рис. 17).

```
purple02@purple-server: ~/RKP/server6
received from 192.168.1.11:5001: CALC_REQUEST;1001;0;444.0;214341.0;555.0;0
checking active clients before calculation...
sent to Client-1 192.168.1.11:5001: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-2 192.168.1.11:5002: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-3 192.168.1.11:5003: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-4 192.168.1.11:5004: STATUS_CHECK;1001;0;0;0;0;0
status response received from 192.168.1.11:5001: STATUS_OK;1001;0;0;0;0;0
client is active: Client-1 192.168.1.11:5001
status response received from 192.168.1.11:5002: STATUS_OK;1001;0;0;0;0;0
client is active: Client-2 192.168.1.11:5002
status response received from 192.168.1.11:5003: STATUS_OK;1001;0;0;0;0;0
client is active: Client-3 192.168.1.11:5003
status response received from 192.168.1.11:5004: STATUS_OK;1001;0;0;0;0;0
client is active: Client-4 192.168.1.11:5004
active clients: 4/4

new calc request: requestId=1001, mainUser=Client-1 192.168.1.11:5001, calcUsers=4, a=444.0, b=214341.0, h=555.0, received=0/4, totalResult=0.0
distributing task between 4 active clients...

sent to Client-1 192.168.1.11:5001: CALC_TASK;1001;0;444.0;53918.25;555.0;0
sent to Client-2 192.168.1.11:5002: CALC_TASK;1001;1;53918.25;107392.5;555.0;0
sent to Client-3 192.168.1.11:5003: CALC_TASK;1001;2;107392.5;160866.75;555.0;0
sent to Client-4 192.168.1.11:5004: CALC_TASK;1001;3;160866.75;214341.0;555.0;0

received from 192.168.1.11:5001: CALC_RESULT;1001;0;0;0;0;1853.8972950755033
received result for request 1001, part 0: 1853.8972950755033
progress: 1/4

received from 192.168.1.11:5002: CALC_RESULT;1001;1;0;0;0;4529.944257615398
received result for request 1001, part 1: 4529.944257615398
progress: 2/4

received from 192.168.1.11:5003: CALC_RESULT;1001;2;0;0;0;-2656.2610191685426
received result for request 1001, part 2: -2656.2610191685426
progress: 3/4

received from 192.168.1.11:5004: CALC_RESULT;1001;3;0;0;0;-3670.0645859047227
received result for request 1001, part 3: -3670.0645859047227
progress: 4/4

sent to Client-1 192.168.1.11:5001: FINAL_RESULT;1001;0;0;0;0;57.5159476176359
request completed: 1001
final result: 57.5159476176359
sent final result to: Client-1 192.168.1.11:5001
```

Рисунок 17 - обработка запроса от первого клиента

После получения результата от сервера в клиенте обновилось поле результата (рис. 18).

Функция: $\cos(x^2)$

Добавление элемента

Нижняя граница интегрирования:

Верхняя граница интегрирования:

Шаг интегрирования:

Добавить

Запись в файл

Нормальный Сериализация

Write **WriteSer**

Read **ReadSer**

Удалить **Вычислить**

Очистить **Заполнить**

Таблица элементов

Нижняя граница	Верхняя граница	Шаг	Результат
444.0	214341.0	555.0	57.5159476176359

Рисунок 18 - результат вычисления на первом клиенте

3.4 Тест четвертый – Вычисление интеграла после закрытия запущенных клиентов

Закрыл один из клиентов (рис. 19).

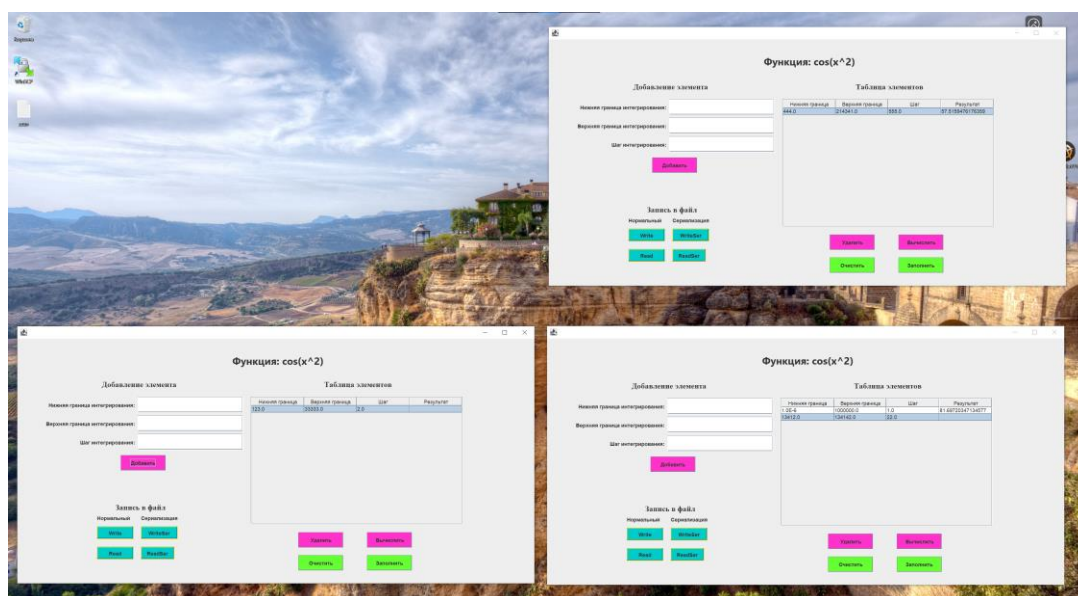


Рисунок 19 - закрытие одного из клиентов

После чего в одном из оставшихся клиентов нажал на вычислить (рис. 20).

Функция: $\cos(x^2)$

Добавление элемента

Нижняя граница интегрирования:

Верхняя граница интегрирования:

Шаг интегрирования:

Добавить

Запись в файл

Нормальный Сериализация

Write WriteSer

Read ReadSer

Таблица элементов

Нижняя граница	Верхняя граница	Шаг	Результат
123.0	33333.0	2.0	

Удалить Вычислить

Очистить Заполнить

Рисунок 20 - запуск вычисления на третьем клиенте

Сервер после получения запроса на вычисление сервер сначала проверил, какие клиенты активны, после чего обработал запрос – распределил вычисления на три оставшихся клиента (рис. 21).

```
purple02@purple-server: ~/RKP/server6
received from 192.168.1.11:5003: CALC_REQUEST;1001;0;123.0;33333.0;2.0;0
checking active clients before calculation...
sent to Client-1 192.168.1.11:5001: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-2 192.168.1.11:5002: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-3 192.168.1.11:5003: STATUS_CHECK;1001;0;0;0;0;0
sent to Client-4 192.168.1.11:5004: STATUS_CHECK;1001;0;0;0;0;0
status response received from 192.168.1.11:5001: STATUS_OK;1001;0;0;0;0;0
client is active: Client-1 192.168.1.11:5001
status response received from 192.168.1.11:5003: STATUS_OK;1001;0;0;0;0;0
client is active: Client-3 192.168.1.11:5003
status response received from 192.168.1.11:5004: STATUS_OK;1001;0;0;0;0;0
client is active: Client-4 192.168.1.11:5004
inactive clients:
Client-2 192.168.1.11:5002
active clients: 3/4
client removed: Client-2 192.168.1.11:5002
new calc request: requestId=1001, mainUser=Client-3 192.168.1.11:5003, calcUsers=3, a=123.0, b=33333.0, h=2.0, received=0/3, totalResult=0.0
distributing task between 3 active clients...
sent to Client-1 192.168.1.11:5001: CALC_TASK;1001;0;123.0;11193.0;2.0;0
sent to Client-3 192.168.1.11:5003: CALC_TASK;1001;1;11193.0;22263.0;2.0;0
sent to Client-4 192.168.1.11:5004: CALC_TASK;1001;2;22263.0;33333.0;2.0;0
received from 192.168.1.11:5003: CALC_RESULT;1001;1;0;0;0;-8.914189324883885
received result for request 1001, part 1: -8.914189324883885
progress: 1/3
received from 192.168.1.11:5001: CALC_RESULT;1001;0;0;0;0;-130.7616649306136
received result for request 1001, part 0: -130.7616649306136
progress: 2/3
received from 192.168.1.11:5004: CALC_RESULT;1001;2;0;0;0;8.251728313004334
received result for request 1001, part 2: 8.251728313004334
progress: 3/3
sent to Client-3 192.168.1.11:5003: FINAL_RESULT;1001;0;0;0;0;-131.42412594249316
request completed: 1001
final result: -131.42412594249316
sent final result to: Client-3 192.168.1.11:5003
```

Рисунок 21 - обработка вычисления для третьего клиента

После получения итогового результата от сервера в третьем клиенте обновилось поле результата (рис. 22).

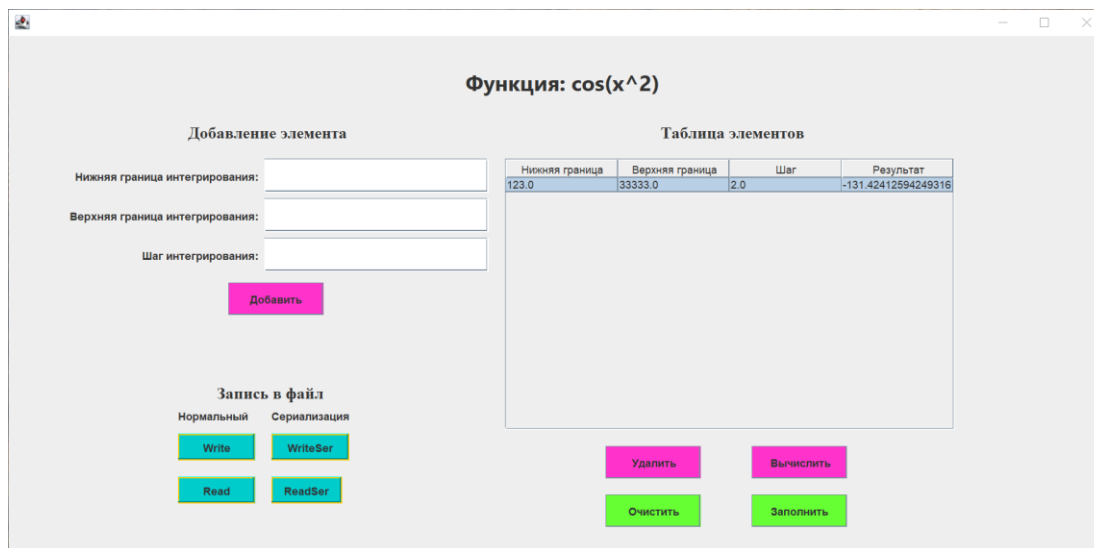


Рисунок 22 - отображение результата в третьем клиенте

Закрыл ещё один клиент (рис. 23).

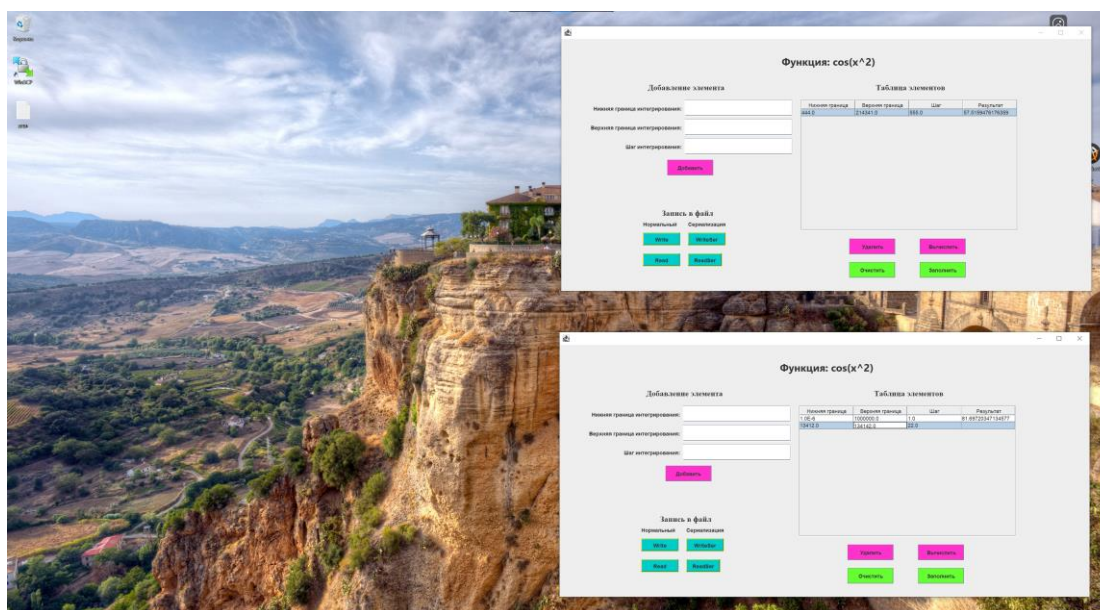


Рисунок 23 - активные клиенты

В одном из оставшихся клиентов нажал «Вычислить» (рис. 24).

Функция: $\cos(x^2)$

Добавление элемента

Нижняя граница интегрирования:

Верхняя граница интегрирования:

Шаг интегрирования:

Добавить

Запись в файл

Нормальный Сериализация

Write WriteSer

Read ReadSer

Таблица элементов

Нижняя граница	Верхняя граница	Шаг	Результат
1.0E-6	1000000.0	1.0	81.69720347134577
13412.0	134142.0	22.0	

Удалить Вычислить

Очистить Заполнить

Рисунок 24 - запуск вычисления на четвертом клиенте

После получения запроса сервер проверил активных пользователей и обработал запрос уже на двух клиентах (рис. 25).

```
purple02@purple-server: ~/RKP/server6

received from 192.168.1.11:5004: CALC_REQUEST;1002;0;13412.0;134142.0;22.0;0
checking active clients before calculation...

sent to Client-1 192.168.1.11:5001: STATUS_CHECK;1002;0;0;0;0;0
sent to Client-3 192.168.1.11:5003: STATUS_CHECK;1002;0;0;0;0;0
sent to Client-4 192.168.1.11:5004: STATUS_CHECK;1002;0;0;0;0;0
status response received from 192.168.1.11:5004: STATUS_OK;1002;0;0;0;0;0
client is active: Client-4 192.168.1.11:5004
status response received from 192.168.1.11:5001: STATUS_OK;1002;0;0;0;0;0
client is active: Client-1 192.168.1.11:5001

inactive clients:
Client-3 192.168.1.11:5003

active clients: 2/3

client removed: Client-3 192.168.1.11:5003

new calc request: requestId=1002, mainUser=Client-4 192.168.1.11:5004, calcUsers=2, a=13412.0, b=134142.0, h=22.0, received=0/2, totalResult=0.0
distributing task between 2 active clients...

sent to Client-4 192.168.1.11:5004: CALC_TASK;1002;0;13412.0;73777.0;22.0;0
sent to Client-1 192.168.1.11:5001: CALC_TASK;1002;1;73777.0;134142.0;22.0;0

received from 192.168.1.11:5004: CALC_RESULT;1002;0;0;0;0;-1314.0725492635165
received result for request 1002, part 0: -1314.0725492635165
progress: 1/2

received from 192.168.1.11:5001: CALC_RESULT;1002;1;0;0;0;-1085.5849119854724
received result for request 1002, part 1: -1085.5849119854724
progress: 2/2

sent to Client-4 192.168.1.11:5004: FINAL_RESULT;1002;0;0;0;0;-2399.657461248989
request completed: 1002
final result: -2399.657461248989
sent final result to: Client-4 192.168.1.11:5004
```

Рисунок 25 - обработка вычисления для четвертого клиента

После получения ответа от сервера в четвертом клиенте обновилось поле результата (рис. 26).

Функция: $\cos(x^2)$

Добавление элемента

Нижняя граница интегрирования:

Верхняя граница интегрирования:

Шаг интегрирования:

Добавить

Запись в файл

Нормальный Сериализация

Write WriteSer

Read ReadSer

Нижняя граница	Верхняя граница	Шаг	Результат
1.0E-6	1000000.0	1.0	81.69720347134577
13412.0	134142.0	22.0	-2399.657461248989

Удалить Вычислить

Очистить Заполнить

Рисунок 26 - отображение результата на четвертом клиенте

Все тесты пройдены успешно, приложение работает корректно.

Вывод: в ходе выполнения лабораторной работы научились создавать клиент-серверные приложения с использованием стандартных классов Java.

Приложение 1 – исходный код приложения-клиента

Приложение 1.1 – содержимое файла «Prog6.java»

```
package com.purpl.prog6;

public class Prog6 {
    public static void main(String[] args) {
        javax.swing.SwingUtilities.invokeLater(() -> new JFrameGUI().setVisible(true));
    }
}
```

Приложение 1.2 – содержимое файла «Integration.java»

```
package com.purpl.prog6;
import java.io.Serializable;

public final class Integration{
    private Integration() {}

    public static class ResIntegral implements Serializable{
        double MIN = 0.000001;
        double MAX = 1000000.0;

        private double a;
        private double b;
        private double h;
        private Double result;

        public ResIntegral(double a, double b, double h) throws OutOfLimitException {
            if(a < MIN || b > MAX) {
                throw new OutOfLimitException("Неверные значения а и b - числа должно
быть в диапазоне: 0,000001-1000000");
            }

            this.a = a;
            this.b = b;
            this.h = h;
        }
        public double getA(){
            return this.a;
        }
        public double getB(){
            return this.b;
        }
        public double getH(){
            return this.h;
        }

        public Double getRes(){
            return this.result;
        }

        public void setA(double num){
            this.a = num;
        }
        public void setB(double num){
            this.b = num;
        }
        public void setH(double num){
```

```

        this.h = num;
    }
    public void setRes(double num){
        this.result = num;
    }
}

public static double integrateSin(double a, double b, double h) {
    if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(h)) {
        throw new IllegalArgumentException("Параметры должны быть конечными
числами.");
    }
    if (h <= 0) throw new IllegalArgumentException("Шаг должен быть > 0.");
    if (a == b) return 0.0;

    double sign = 1.0;
    if (a > b) {
        double tmp = a; a = b; b = tmp;
        sign = -1.0;
    }

    double sum = 0.0;
    double x = a;

    while (x < b) {
        double x2 = Math.min(x + h, b);
        sum += (Math.cos(x*x) + Math.cos(x2*x2)) * 0.5 * (x2 - x);
        x = x2;
    }
    return sign * sum;
}
}

```

Приложение 1.3 – содержимое файла «JFrameGUI.java»

```
package com.purpl.prog6;

import java.util.ArrayList;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import javax.swing.filechooser.FileNameExtensionFilter;
import javax.swing.JFileChooser;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.Locale;
import java.util.HashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.atomic.AtomicInteger;

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this
template
 */

/**
 *
 * @author izada
 *
 *
 */
public class JFrameGUI extends javax.swing.JFrame {

    private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(JFrameGUI.class.getName());

    private final ArrayList<Integration.ResIntegral> entries = new ArrayList<>();
    private DefaultTableModel tableModel;
    private boolean isCleared = false;
    private volatile boolean calcInProgress = false;
    private static final int THREAD_COUNT = 9;

    private ClientUdpService udpService;
    private final AtomicInteger requestCounter = new AtomicInteger(1000);
    private final HashMap<Integer, Integer> requestRows = new HashMap<>();

    private static final String SERVER_HOST = "192.168.1.51";
    private static final int SERVER_PORT = 5000;
```

```

/**
 * Creates new form JFrameGUI
 */
public JFrameGUI() {
    initComponents();

    tableModel = new DefaultTableModel(
        new Object[]{"Нижняя граница", "Верхняя граница", "Шаг", "Результат"},
0) {
        @Override public boolean isCellEditable(int row, int col) {
            if (col == 3) {
                return false;
            } else {
                return true;
            }
        }
    };
    jTableVariables.setModel(tableModel);

    initUdpClient();

    addWindowListener(new java.awt.event.WindowAdapter() {
        @Override
        public void windowClosing(java.awt.event.WindowEvent e) {
            if (udpService != null) {
                udpService.stop();
            }
        }
    });
}

private void initUdpClient() {
    try {
        udpService = new ClientUdpService(this, SERVER_HOST, SERVER_PORT);
        udpService.start();
        udpService.registerOnServer();

        JOptionPane.showMessageDialog(this,
            "UDP-клиент запущен на порту: " + udpService.getLocalPort(),
            "Сеть", JOptionPane.INFORMATION_MESSAGE);

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this,
            "Ошибка запуска UDP-клиента: " + ex.getMessage(),
            "Сеть", JOptionPane.WARNING_MESSAGE);
    }
}

```

```

/**
 * This method is called from within the constructor to initialize the form.
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jPanelMain = new javax.swing.JPanel();
    jTextFieldUpperLimit = new javax.swing.JTextField();
    jTextFieldStep = new javax.swing.JTextField();
    jTextFieldLowerLimit = new javax.swing.JTextField();
    jLabelLowerLimit = new javax.swing.JLabel();
    jLabelUpperLimit = new javax.swing.JLabel();
    jLabelStep = new javax.swing.JLabel();
    jScrollPane2 = new javax.swing.JScrollPane();
    jTableVariables = new javax.swing.JTable();
    jButtonAdd = new javax.swing.JButton();
    jButtonDelete = new javax.swing.JButton();
    jButtonCalc = new javax.swing.JButton();
    jLabelMainHeader = new javax.swing.JLabel();
    jButtonClear = new javax.swing.JButton();
    jButtonFill = new javax.swing.JButton();
    jButtonWrite = new javax.swing.JButton();
    jButtonWriteSer = new javax.swing.JButton();
    jButtonRead = new javax.swing.JButton();
    jButtonReadSer = new javax.swing.JButton();
    jLabelLowerLimit1 = new javax.swing.JLabel();
    jLabelLowerLimit2 = new javax.swing.JLabel();
    jLabel1 = new javax.swing.JLabel();
    jLabel2 = new javax.swing.JLabel();
    jLabel3 = new javax.swing.JLabel();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

    jLabelLowerLimit.setText("Нижняя граница интегрирования:");

    jLabelUpperLimit.setText("Верхняя граница интегрирования:");

    jLabelStep.setText("Шаг интегрирования:");

    jTableVariables.setModel(new javax.swing.table.DefaultTableModel(
        new Object [][] {
            {null, null, null, null},
            {null, null, null, null},
            {null, null, null, null},
            {null, null, null, null}
        },
        new String [] {

```

```

        "Нижняя граница", "Верхняя граница", "Шаг", "Результат"
    }
));
jScrollPane2.setViewportViewView(jTableVariables);

jButtonAdd.setBackground(new java.awt.Color(255, 51, 204));
jButtonAdd.setText("Добавить");
jButtonAdd.addActionListener(this::jButtonAddActionPerformed);

jButtonDelete.setBackground(new java.awt.Color(255, 51, 204));
jButtonDelete.setText("Удалить");
jButtonDelete.addActionListener(this::jButtonDeleteActionPerformed);

jButtonCalc.setBackground(new java.awt.Color(255, 51, 204));
jButtonCalc.setText("Вычислить");
jButtonCalc.addActionListener(this::jButtonCalcActionPerformed);

jLabelMainHeader.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
jLabelMainHeader.setText("Функция:  $\cos(x^2)$ ");

jButtonClear.setBackground(new java.awt.Color(102, 255, 51));
jButtonClear.setText("Очистить");
jButtonClear.addActionListener(this::jButtonClearActionPerformed);

jButtonFill.setBackground(new java.awt.Color(102, 255, 51));
jButtonFill.setText("Заполнить");
jButtonFill.addActionListener(this::jButtonFillActionPerformed);

jButtonWrite.setBackground(new java.awt.Color(0, 204, 204));
jButtonWrite.setText("Write");

jButtonWrite.setBorder(javax.swing.BorderFactory.createBevelBorder(javax.swing.border.BevelBorder.RAISED, null, java.awt.Color.orange, java.awt.Color.orange, null));
jButtonWrite.setCursor(new java.awt.Cursor(java.awt.Cursor.DEFAULT_CURSOR));
jButtonWrite.addActionListener(this::jButtonWriteActionPerformed);

jButtonWriteSer.setBackground(new java.awt.Color(0, 204, 204));
jButtonWriteSer.setText("WriteSer");

jButtonWriteSer.setBorder(javax.swing.BorderFactory.createBevelBorder(javax.swing.border.BevelBorder.RAISED, null, java.awt.Color.orange, java.awt.Color.orange, null));
jButtonWriteSer.addActionListener(this::jButtonWriteSerActionPerformed);

jButtonRead.setBackground(new java.awt.Color(0, 204, 204));
jButtonRead.setText("Read");

jButtonRead.setBorder(javax.swing.BorderFactory.createBevelBorder(javax.swing.border.BevelBorder.RAISED, null, java.awt.Color.orange, java.awt.Color.orange, null));
jButtonRead.addActionListener(this::jButtonReadActionPerformed);

```



```
jButtonReadSer.setBackground(new java.awt.Color(0, 204, 204));  
jButtonReadSer.setText("ReadSer");  
  
jButtonReadSer.setBorder(javax.swing.BorderFactory.createBevelBorder(javax.swing.border.  
BevelBorder.RAISED, null, java.awt.Color.orange, java.awt.Color.orange, null));  
jButtonReadSer.addActionListener(this::jButtonReadSerActionPerformed);  
  
jLabelLowerLimit1.setText("Сериализация");  
  
jLabelLowerLimit2.setText("Нормальный");  
  
jLabel1.setFont(new java.awt.Font("Times New Roman", 1, 18)); // NOI18N  
jLabel1.setText("Запись в файл");  
  
jLabel2.setFont(new java.awt.Font("Times New Roman", 1, 18)); // NOI18N  
jLabel2.setText("Добавление элемента");  
  
jLabel3.setFont(new java.awt.Font("Times New Roman", 1, 18)); // NOI18N  
jLabel3.setText("Таблица элементов");  
  
javax.swing.GroupLayout jPanelMainLayout = new  
javax.swing.GroupLayout(jPanelMain);  
jPanelMain.setLayout(jPanelMainLayout);  
jPanelMainLayout.setHorizontalGroup(  
  
jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
.addGroup(jPanelMainLayout.createSequentialGroup()  
    .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGroup(jPanelMainLayout.createSequentialGroup()  
            .addGroup(jPanelMainLayout.createSequentialGroup().addGap(56, 56, 56).addGap(56, 56, 56))  
            .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)  
                .addComponent(jLabelLowerLimit) .addComponent(jLabelUpperLimit) .addComponent(jLabelStep))  
            .addGap(6, 6, 6)  
            .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
                .addComponent(jTextFieldUpperLimit,  
                    javax.swing.GroupLayout.PREFERRED_SIZE, 250, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addComponent(jTextFieldLowerLimit,  
                    javax.swing.GroupLayout.PREFERRED_SIZE, 250, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addComponent(jTextFieldStep,  
                    javax.swing.GroupLayout.PREFERRED_SIZE, 250, javax.swing.GroupLayout.PREFERRED_SIZE)))  
            .addGroup(jPanelMainLayout.createSequentialGroup().addGap(177, 177, 177).
```

```

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)

        .addGroup(jPanelMainLayout.createSequentialGroup()

                .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)

                        .addComponent(jButtonRead,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

                        .addComponent(jLabelLowerLimit2)
                        .addComponent(jButtonWrite,
javax.swing.GroupLayout.PREFERRED_SIZE, 86, javax.swing.GroupLayout.PREFERRED_SIZE))
                        .addGap(18, 18, 18)

                .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)

                        .addComponent(jLabelLowerLimit1,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

                        .addComponent(jButtonWriteSer,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

                        .addComponent(jButtonReadSer,
javax.swing.GroupLayout.PREFERRED_SIZE, 79, javax.swing.GroupLayout.PREFERRED_SIZE)))
                .addGroup(jPanelMainLayout.createSequentialGroup()

                        .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
G)

                                .addComponent(jButtonAdd,
javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE)
                                .addComponent(jLabel1))
                                .addGap(29, 29, 29))))
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 18,
Short.MAX_VALUE)

                .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING
)

                        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanelMainLayout.createSequentialGroup()

                                .addComponent(jScrollPane2,
javax.swing.GroupLayout.PREFERRED_SIZE, 501, javax.swing.GroupLayout.PREFERRED_SIZE)
                                .addGap(39, 39, 39))

                        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanelMainLayout.createSequentialGroup()

                                .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING
)

                                        .addComponent(jButtonDelete,
javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE)

```

```

        .addComponent(jButtonClear,
javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(57, 57, 57)

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING
)
        .addComponent(jButtonFill,
javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jButtonCalc,
javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(159, 159, 159))))
    .addGroup(jPanelMainLayout.createSequentialGroup()
        .addGap(188, 188, 188)
        .addComponent(jLabel2)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jLabel3)
        .addGap(207, 207, 207))
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanelMainLayout.createSequentialGroup()
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jLabelMainHeader)
        .addGap(368, 368, 368))
    );
jPanelMainLayout.setVerticalGroup(

jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanelMainLayout.createSequentialGroup()
        .addGap(23, 23, 23)
        .addComponent(jLabelMainHeader)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 35,
Short.MAX_VALUE)

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELIN
E)
        .addComponent(jLabel2)
        .addComponent(jLabel3))
        .addGap(18, 18, 18)

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING
, false)
        .addGroup(jPanelMainLayout.createSequentialGroup()
            .addComponent(jScrollPane2,
javax.swing.GroupLayout.PREFERRED_SIZE, 302, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(18, 18, 18)

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELIN
E)

```

```

        .addComponent(jButtonCalc,
javax.swing.GroupLayout.PREFERRED_SIZE, 36, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jButtonDelete,
javax.swing.GroupLayout.PREFERRED_SIZE, 36, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jButtonClear,
javax.swing.GroupLayout.PREFERRED_SIZE, 36, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jButtonFill,
javax.swing.GroupLayout.PREFERRED_SIZE, 36, javax.swing.GroupLayout.PREFERRED_SIZE))
    .addGap(14, 14, 14))
    .addGroup(jPanelMainLayout.createSequentialGroup())

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jTextFieldLowerLimit,
javax.swing.GroupLayout.PREFERRED_SIZE, 38, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jLabelLowerLimit))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jTextFieldUpperLimit,
javax.swing.GroupLayout.PREFERRED_SIZE, 38, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jLabelUpperLimit))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jTextFieldStep,
javax.swing.GroupLayout.PREFERRED_SIZE, 38, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jLabelStep))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
    .addComponent(jButtonAdd,
javax.swing.GroupLayout.PREFERRED_SIZE, 36, javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    .addComponent(jLabel1)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jLabelLowerLimit1)

```

```

        .addComponent(jLabelLowerLimit2))

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

    .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
        .addComponent(jButtonWriteSer,
            javax.swing.GroupLayout.PREFERRED_SIZE, 30, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jButtonWrite,
            javax.swing.GroupLayout.PREFERRED_SIZE, 30, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)

    .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jButtonRead,
            javax.swing.GroupLayout.PREFERRED_SIZE, 30, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jButtonReadSer,
            javax.swing.GroupLayout.PREFERRED_SIZE, 30, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(40, 40, 40)))
    );

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jPanelMain, javax.swing.GroupLayout.PREFERRED_SIZE,
                javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addContainerGap(131, Short.MAX_VALUE))
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jPanelMain, javax.swing.GroupLayout.DEFAULT_SIZE,
                javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addGap(17, 17, 17))
    );

    pack();
} // </editor-fold>

private void jButtonAddActionPerformed(java.awt.event.ActionEvent evt) {
    if(isCleared == true)
    {
        try
        {
            entries.clear();
        }
    }
}

```

```

        catch (Exception ex)
        {
            JOptionPane.showMessageDialog(this,
                "Ошибка: " + ex.getMessage(),
                "Удаление данных", JOptionPane.WARNING_MESSAGE);
        }
        isCleared = false;
    }

    try {
        double a = parse(jTextFieldLowerLimit.getText(), "Нижняя граница");
        double b = parse(jTextFieldUpperLimit.getText(), "Верхняя граница");
        double h = parse(jTextFieldStep.getText(), "Шаг");

        if (h <= 0) throw new IllegalArgumentException("Шаг должен быть > 0.");

        Integration.ResIntegral e = new Integration.ResIntegral(a, b, h);
        entries.add(e);

        tableModel.addRow(new Object[]{a, b, h, ""});

        jTextFieldLowerLimit.setText("");
        jTextFieldUpperLimit.setText("");
        jTextFieldStep.setText("");

        int last = tableModel.getRowCount() - 1;
        jTableVariables.setRowSelectionInterval(last, last);

    } catch (OutOfLimitException ex) {
        JOptionPane.showMessageDialog(this,
            ex.getMessage(),
            "Ошибка", JOptionPane.WARNING_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this,
            "Ошибка: " + ex.getMessage(),
            "Ввод данных", JOptionPane.WARNING_MESSAGE);
    }
}

private void jButtonDeleteActionPerformed(java.awt.event.ActionEvent evt) {
    int row = jTableVariables.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Выбери строку в таблице.",
            "Удаление", JOptionPane.INFORMATION_MESSAGE);
        return;
    }
    entries.remove(row);
    tableModel.removeRow(row);
}

```

```

private class IntegralCalcTask implements Runnable {
    private final double a;
    private final double b;
    private final double h;
    private final int row;
    private final int num;

    public IntegralCalcTask(double a, double b, double h, int row, int num) {
        this.a = a;
        this.b = b;
        this.h = h;
        this.row = row;
        this.num = num;
    }

    @Override
    public void run() {
        try {
            System.out.println("Thread " + num);
            Integration.ResIntegral e = new Integration.ResIntegral(a, b, h);
            double res = Integration.integrateSin(e.getA(), e.getB(), e.getH());
            e.setRes(res);

            javax.swing.SwingUtilities.invokeLater(() -> {
                tableModel.setValueAt(res, row, 3);
                if (row < entries.size()) {
                    entries.set(row, e);
                }
            });
        } catch (OutOfLimitException ex) {
            javax.swing.SwingUtilities.invokeLater(() -> {
                JOptionPane.showMessageDialog(JFrameGUI.this,
                    ex.getMessage(),
                    "Ошибка", JOptionPane.WARNING_MESSAGE);
            });
        } catch (Exception ex) {
            javax.swing.SwingUtilities.invokeLater(() -> {
                JOptionPane.showMessageDialog(JFrameGUI.this,
                    "Ошибка вычисления: " + ex.getMessage(),
                    "Ошибка", JOptionPane.WARNING_MESSAGE);
            });
        }
    }
}

private void jButtonCalcActionPerformed(java.awt.event.ActionEvent evt) {
    int row = jTableVariables.getSelectedRow();

    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Выбери строку в таблице.",

```

```

        "Вычисление", JOptionPane.INFORMATION_MESSAGE);
    return;
}

if (udpService == null) {
    JOptionPane.showMessageDialog(this,
        "UDP-клиент не запущен.",
        "Сеть", JOptionPane.WARNING_MESSAGE);
    return;
}

try {
    double A = parse(tableModel.getValueAt(row, 0).toString(), "Нижняя
граница");
    double B = parse(tableModel.getValueAt(row, 1).toString(), "Верхняя
граница");
    double H = parse(tableModel.getValueAt(row, 2).toString(), "Шаг");

    int requestId = requestCounter.incrementAndGet();
    requestRows.put(requestId, row);

    tableModel.setValueAt("ожидание...", row, 3);

    udpService.sendCalcRequest(requestId, A, B, H);

} catch (Exception ex) {
    JOptionPane.showMessageDialog(this,
        "Ошибка отправки запроса: " + ex.getMessage(),
        "Ошибка", JOptionPane.WARNING_MESSAGE);
}
}

private void jButtonClearActionPerformed(java.awt.event.ActionEvent evt) {
    for (int i = tableModel.getRowCount() - 1; i >= 0; i--)
    {
        tableModel.removeRow(i);
    }
    jTableVariables.clearSelection();
    isCleared = true;
}

private void jButtonFillActionPerformed(java.awt.event.ActionEvent evt) {
    tableModel.setRowCount(0);

    for (Integration.ResIntegral e : entries) {
        Object res = (e.getRes() == null) ? "" : e.getRes();
        tableModel.addRow(new Object[]{ e.getA(), e.getB(), e.getH(), res });
    }
    isCleared = false;
}
}

```



```

private void jButtonWriteActionPerformed(java.awt.event.ActionEvent evt) {
    if(isCleared == true)
    {
        try
        {
            entries.clear();
        }
        catch (Exception ex)
        {
            JOptionPane.showMessageDialog(this,
                "Ошибка: " + ex.getMessage(),
                "Удаление данных", JOptionPane.WARNING_MESSAGE);
        }
        isCleared = false;
    }

    if(!entries.isEmpty())
    {
        try
        {
            File file = chooseFile(true, "Сохранить файл", "txt", "Текстовые файлы
(*.txt)");
            if (file == null) return;

            writeFile(file, entries);
            JOptionPane.showMessageDialog(this, "Файл сохранён:\n" +
file.getAbsolutePath(),
                "Запись", JOptionPane.INFORMATION_MESSAGE);
        }
        catch(Exception ex)
        {
            JOptionPane.showMessageDialog(this,
                "Ошибка записи: " + ex.getMessage());
        }
    }
    else
    {
        JOptionPane.showMessageDialog(this,
            "Ошибка записи: лист entries пуст");
    }
}

private void jButtonWriteSerActionPerformed(java.awt.event.ActionEvent evt) {
    if(isCleared == true)
    {
        try
        {
            entries.clear();

```

```

    }
    catch (Exception ex)
    {
        JOptionPane.showMessageDialog(this,
            "Ошибка: " + ex.getMessage(),
            "Удаление данных", JOptionPane.WARNING_MESSAGE);
    }
    isCleared = false;
}

if(!entries.isEmpty())
{
    try
    {
        File file = chooseFile(true, "Сохранить файл", "txt", "Текстовые файлы (*.txt)");
        if (file == null) return;

        writeSer(file, entries);
        JOptionPane.showMessageDialog(this, "Файл сохранён:\n" +
file.getAbsolutePath(),
            "Запись", JOptionPane.INFORMATION_MESSAGE);
    }
    catch(Exception ex)
    {
        JOptionPane.showMessageDialog(this,
            "Ошибка записи: " + ex.getMessage());
    }
}
else
{
    JOptionPane.showMessageDialog(this,
        "Ошибка записи: лист entries пуст");
}
}

private void jButtonReadActionPerformed(java.awt.event.ActionEvent evt) {
    entries.clear();
    File file = chooseFile(false, "Открыть файл", "txt", "Текстовые файлы (*.txt)");
    if (file == null) return;

    try {
        var list = readFile(file);
        loadEntriesToTable(list);
        JOptionPane.showMessageDialog(this, "Прочитано:\n" +
file.getAbsolutePath());
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Ошибка чтения: " + ex.getMessage());
    }
}

```

```

    }

    private void jButtonReadSerActionPerformed(java.awt.event.ActionEvent evt) {
        entries.clear();
        File file = chooseFile(false, "Открыть файл", "txt", "Текстовые файлы (*.txt)");
        if (file == null) return;

        try {
            var list = readSer(file);
            loadEntriesToTable(list);
            JOptionPane.showMessageDialog(this, "Прочитано:\n" +
file.getAbsolutePath());
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Ошибка чтения: " + ex.getMessage());
        }
    }

    private double parse(String s, String fieldName) {
        s = s.trim();
        if (s.isEmpty()) throw new IllegalArgumentException(fieldName + " не задано.");
        s = s.replace(',', '.', '.');
        return Double.parseDouble(s);
    }

    private void writeFile(File file, ArrayList<Integration.ResIntegral> list)
    {
        BufferedWriter bw = null;
        try
        {
            bw = new BufferedWriter(new FileWriter(file, StandardCharsets.UTF_8));
            bw.write("{}");
            bw.newLine();
            int count = 0;
            for(Integration.ResIntegral e : list)
            {
                String a = String.format(Locale.US, "%.10f", e.getA());
                String b = String.format(Locale.US, "%.10f", e.getB());
                String h = String.format(Locale.US, "%.10f", e.getH());
                String res;
                if(e.getRes() != null)
                {
                    res = String.format(Locale.US, "%.10f", e.getRes());
                }
                else
                {
                    res = "n";
                }
                bw.write("\"" + count + "\" : {" + "\"a\":" + a + "," + "\"b\":" + b +
",\" + "\"h\":" + h + "," + "\"res\":" + res + "},"");
                count++;
            }
        }
        catch (IOException ex) {
            JOptionPane.showMessageDialog(this, "Ошибка записи: " + ex.getMessage());
        }
        finally {
            if (bw != null) bw.close();
        }
    }

```

```

        bw.newLine();
    }
    bw.write("{}");
    bw.flush();
    bw.close();
}
catch (IOException ex)
{
    ex.printStackTrace();
}
}

private ArrayList<Integration.ResIntegral> readFile(File file)
{
    ArrayList<Integration.ResIntegral> list = new ArrayList<>();
    BufferedReader br = null;
    try
    {
        br = new BufferedReader(new FileReader(file, StandardCharsets.UTF_8));
        String line;
        line = br.readLine();
        int i = 0;
        while(!(line = br.readLine()).equals("{}"))
        {
            i++;
            String[] p = line.split(" : ");

            String cou = p[0];
            String znach = p[1];

            znach = znach.replace("{", "");
            znach = znach.replace("}", "", "");

            String[] pp = znach.split(",");

            String aa = pp[0];
            String bb = pp[1];
            String hh = pp[2];
            String resres = pp[3];

            String[] A = aa.split(":");
            String[] B = bb.split(":");
            String[] H = hh.split(":");
            String[] RES = resres.split(":");

            double a = parse(A[1], "Нижняя граница");
            double b = parse(B[1], "Верхняя граница");
            double h = parse(H[1], "Шаг");

            Integration.ResIntegral e = new Integration.ResIntegral(a, b, h);

```

```

        if (!RES[1].equals("\n\"")){
            String resStr = RES[1].trim();
            resStr = resStr.replace(',', '.');
            e.setRes(Double.parseDouble(resStr));
        }
        list.add(e);
    }
}
catch(IOException ex)
{
    ex.printStackTrace();
}
catch(OutOfLimitException e)
{
    e.printStackTrace();
}
return list;
}

private void writeSer(File file, ArrayList<Integration.ResIntegral> list)
{
    ObjectOutputStream out = null;

    try
    {
        out = new ObjectOutputStream(new BufferedOutputStream(new
FileOutputStream(file)));
        out.writeObject(list);
        out.close();
    }
    catch(IOException ex)
    {
        ex.printStackTrace();
    }
}

private ArrayList<Integration.ResIntegral> readSer(File file)
{
    ObjectInputStream in = null;
    ArrayList<Integration.ResIntegral> obj = null;
    try
    {
        in = new ObjectInputStream(new BufferedInputStream(new
FileInputStream(file)));
        obj = (ArrayList<Integration.ResIntegral>)in.readObject();
        in.close();
        return (ArrayList<Integration.ResIntegral>) obj;
    }
}

```

```

        catch(IOException ex)
        {
            ex.printStackTrace();
        }
        catch(ClassNotFoundException e)
        {
            e.printStackTrace();
        }
        return new ArrayList<>();
    }

    private void loadEntriesToTable(ArrayList<Integration.ResIntegral> list)
    {
        entries.clear();
        entries.addAll(list);

        tableModel.setRowCount(0);
        for (Integration.ResIntegral e : entries)
        {
            Object res = (e.getRes() == null) ? "" : e.getRes();
            tableModel.addRow(new Object[]{e.getA(), e.getB(), e.getH(), res});
        }

        isCleared = false;
        jTableVariables.clearSelection();
    }

    private File chooseFile(boolean save, String title, String extension, String
description)
    {
        JFileChooser fc = new JFileChooser();
        fc.setDialogTitle(title);
        fc.setFileFilter(new FileNameExtensionFilter(description, extension));

        int res = save ? fc.showSaveDialog(this) : fc.showOpenDialog(this);
        if (res != JFileChooser.APPROVE_OPTION) return null;

        File f = fc.getSelectedFile();

        return f;
    }

    public void receiveFinalResult(int requestId, double result) {
        javax.swing.SwingUtilities.invokeLater(() -> {
            Integer row = requestRows.get(requestId);

            if (row == null) {
                JOptionPane.showMessageDialog(this,
                    "Получен результат для неизвестного запроса: " + requestId,

```

```

        "Сеть", JOptionPane.WARNING_MESSAGE);
    return;
}

tableModel.setValueAt(result, row, 3);

try {
    double a = parse(tableModel.getValueAt(row, 0).toString(), "Нижняя
граница");
    double b = parse(tableModel.getValueAt(row, 1).toString(), "Верхняя
граница");
    double h = parse(tableModel.getValueAt(row, 2).toString(), "Шаг");

    Integration.ResIntegral e = new Integration.ResIntegral(a, b, h);
    e.setRes(result);

    if (row < entries.size()) {
        entries.set(row, e);
    }

} catch (Exception ex) {
    JOptionPane.showMessageDialog(this,
        "Результат получен, но не удалось обновить коллекцию: " +
ex.getMessage(),
        "Сеть", JOptionPane.WARNING_MESSAGE);
}

requestRows.remove(requestId);
});
}

public void calculateServerTask(Message msg) {
    new Thread(() -> {
        try {
            double result = calculatePartInThreads(msg.getA(), msg.getB(),
msg.getH());

            udpService.sendCalcResult(
                msg.getRequestId(),
                msg.getPartNum(),
                result
            );

        } catch (Exception ex) {
            showNetworkError("Ошибка вычисления части интеграла: " +
ex.getMessage());
        }
    }).start();
}
}

```

```

private double calculatePartInThreads(double a, double b, double h) throws Exception
{
    double[] results = new double[THREAD_COUNT];
    CountDownLatch latch = new CountDownLatch(THREAD_COUNT);

    for (int i = 0; i < THREAD_COUNT; i++) {
        final int index = i;

        Thread t = new Thread(() -> {
            try {
                System.out.println("Client thread " + index);

                Integration.ResIntegral e = new Integration.ResIntegral(a, b, h);
                double res = Integration.integrateSin(e.getA(), e.getB(), e.getH());

                results[index] = res;

                System.out.println("Client thread " + index
                    + " calculated full part ["
                    + a + "; " + b + "] = " + res);

            } catch (Exception ex) {
                System.out.println("Client thread " + index
                    + " error: " + ex.getMessage());
            } finally {
                latch.countDown();
            }
        });

        t.start();
    }

    latch.await();

    return results[0];
}

public void showNetworkError(String text) {
    javax.swing.SwingUtilities.invokeLater(() -> {
        JOptionPane.showMessageDialog(this,
            text,
            "Сетевая ошибка", JOptionPane.WARNING_MESSAGE);
    });
}

public void showNetworkStatus(String text) {
    javax.swing.SwingUtilities.invokeLater(() -> {
        System.out.println(text);
    });
}

```



```
// Variables declaration - do not modify
private javax.swing.JButton jButtonAdd;
private javax.swing.JButton jButtonCalc;
private javax.swing.JButton jButtonClear;
private javax.swing.JButton jButtonDelete;
private javax.swing.JButton jButtonFill;
private javax.swing.JButton jButtonRead;
private javax.swing.JButton jButtonReadSer;
private javax.swing.JButton jButtonWrite;
private javax.swing.JButton jButtonWriteSer;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabelLowerLimit;
private javax.swing.JLabel jLabelLowerLimit1;
private javax.swing.JLabel jLabelLowerLimit2;
private javax.swing.JLabel jLabelMainHeader;
private javax.swing.JLabel jLabelStep;
private javax.swing.JLabel jLabelUpperLimit;
private javax.swing.JPanel jPanelMain;
private javax.swing.JScrollPane jScrollPane2;
private javax.swing.JTable jTableVariables;
private javax.swing.JTextField jTextFieldLowerLimit;
private javax.swing.JTextField jTextFieldStep;
private javax.swing.JTextField jTextFieldUpperLimit;
// End of variables declaration
}
```

Приложение 1.4 – содержимое файла «OutOfLimitException.java»

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
 this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
 template
 */
package com.purpl.prog6;

/**
 *
 * @author izada
 */
public class OutOfLimitException extends Exception {
    public OutOfLimitException(String message){
        super(message);
    }
}
```

Приложение 1.5 – содержимое файла «Message.java»

```
package com.purpl.prog6;

public class Message {
    public static final String TYPE_REGISTER = "REGISTER";
    public static final String TYPE_REGISTER_OK = "REGISTER_OK";
    public static final String TYPE_CALC_REQUEST = "CALC_REQUEST";
    public static final String TYPE_CALC_TASK = "CALC_TASK";
    public static final String TYPE_CALC_RESULT = "CALC_RESULT";
    public static final String TYPE_FINAL_RESULT = "FINAL_RESULT";
    public static final String TYPE_ERROR = "ERROR";

    public static final String TYPE_STATUS_CHECK = "STATUS_CHECK";
    public static final String TYPE_STATUS_OK = "STATUS_OK";

    private final String rawText;
    private final String msgType;
    private final int requestId;
    private final int partNum;
    private final double a;
    private final double b;
    private final double h;
    private final double res;

    public Message(String rawText) {
        this.rawText = rawText.trim();

        String[] parts = this.rawText.split(";");

        if (parts.length < 7) {
            throw new IllegalArgumentException("wrong message format: " + rawText);
        }

        this.msgType = parts[0].trim();
        this.requestId = Integer.parseInt(parts[1].trim());
        this.partNum = Integer.parseInt(parts[2].trim());
        this.a = Double.parseDouble(parts[3].trim());
        this.b = Double.parseDouble(parts[4].trim());
        this.h = Double.parseDouble(parts[5].trim());
        this.res = Double.parseDouble(parts[6].trim());
    }

    public String getMsgType() {
        return msgType;
    }

    public int getRequestId() {
```

```

        return requestId;
    }

    public int getPartNum() {
        return partNum;
    }

    public double getA() {
        return a;
    }

    public double getB() {
        return b;
    }

    public double getH() {
        return h;
    }

    public double getRes() {
        return res;
    }

    public boolean isRegisterOk() {
        return TYPE_REGISTER_OK.equals(msgType);
    }

    public boolean isCalcTask() {
        return TYPE_CALC_TASK.equals(msgType);
    }

    public boolean isFinalResult() {
        return TYPE_FINAL_RESULT.equals(msgType);
    }

    public boolean isError() {
        return TYPE_ERROR.equals(msgType);
    }

    public boolean isStatusCheck() {
        return TYPE_STATUS_CHECK.equals(msgType);
    }

    public static String createRegister() {
        return TYPE_REGISTER + ";0;0;0;0;0;0";
    }

    public static String createCalcRequest(int requestId, double a, double b, double h)
{
        return TYPE_CALC_REQUEST + ";"

```

```

        + requestId + ";"
        + 0 + ";"
        + a + ";"
        + b + ";"
        + h + ";"
        + 0;
    }

    public static String createCalcResult(int requestId, int partNum, double result) {
        return TYPE_CALC_RESULT + ";"
            + requestId + ";"
            + partNum + ";"
            + 0 + ";"
            + 0 + ";"
            + 0 + ";"
            + result;
    }

    public static String createStatusOk(int requestId) {
        return TYPE_STATUS_OK + ";"
            + requestId + ";"
            + 0 + ";"
            + 0 + ";"
            + 0 + ";"
            + 0 + ";"
            + 0;
    }

    @Override
    public String toString() {
        return rawText;
    }
}

```

Приложение 1.6 – содержимое файла «ClientUdpService.java»

```
package com.purpl.prog6;

import java.io.IOException;
import java.net.BindException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.nio.charset.StandardCharsets;

public class ClientUdpService {
    private static final int CLIENT_PORT_FROM = 5001;
    private static final int CLIENT_PORT_TO = 5010;

    private final JFrameGUI gui;
    private final InetAddress serverAddress;
    private final int serverPort;

    private DatagramSocket socket;
    private volatile boolean running = false;

    public ClientUdpService(JFrameGUI gui, String serverHost, int serverPort) throws
Exception {
        this.gui = gui;
        this.serverAddress = InetAddress.getByName(serverHost);
        this.serverPort = serverPort;
        this.socket = createSocketInRange(CLIENT_PORT_FROM, CLIENT_PORT_TO);
    }

    private DatagramSocket createSocketInRange(int from, int to) throws IOException {
        for (int port = from; port <= to; port++) {
            try {
                return new DatagramSocket(port);
            } catch (BindException ex) {
                // порт занят, пробуем следующий
            }
        }

        throw new IOException("Нет свободных UDP-портов в диапазоне " + from + "-" +
to);
    }

    public int getLocalPort() {
        return socket.getLocalPort();
    }

    public void start() {
```

```

        running = true;

        Thread listenerThread = new Thread(this::listenLoop);
        listenerThread.setDaemon(true);
        listenerThread.start();
    }

    public void stop() {
        running = false;

        if (socket != null && !socket.isClosed()) {
            socket.close();
        }
    }

    public void registerOnServer() throws IOException {
        sendToServer(Message.createRegister());
    }

    public void sendCalcRequest(int requestId, double a, double b, double h) throws
IOException {
        String message = Message.createCalcRequest(requestId, a, b, h);
        sendToServer(message);
    }

    public void sendCalcResult(int requestId, int partNum, double result) throws
IOException {
        String message = Message.createCalcResult(requestId, partNum, result);
        sendToServer(message);
    }

    public void sendStatusOk(int requestId) throws IOException {
        String message = Message.createStatusOk(requestId);
        sendToServer(message);
    }

    private void sendToServer(String message) throws IOException {
        byte[] data = message.getBytes(StandardCharsets.UTF_8);

        DatagramPacket packet = new DatagramPacket(
            data,
            data.length,
            serverAddress,
            serverPort
        );

        socket.send(packet);

        System.out.println("sent to server: " + message);
    }

```

```

private void listenLoop() {
    byte[] buffer = new byte[1024];

    while (running) {
        try {
            DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
            socket.receive(packet);

            String text = new String(
                packet.getData(),
                0,
                packet.getLength(),
                StandardCharsets.UTF_8
            );

            System.out.println("received from server: " + text);

            Message msg = new Message(text);
            handleMessage(msg);

        } catch (Exception ex) {
            if (running) {
                gui.showNetworkError("Ошибка получения UDP-сообщения: " +
ex.getMessage());
            }
        }
    }
}

private void handleMessage(Message msg) throws Exception {
    if (msg.isRegisterOk()) {
        gui.showNetworkStatus("Клиент зарегистрирован на сервере. Локальный UDP-
порт: " + getLocalPort());

    } else if (msg.isStatusCheck()) {
        sendStatusOk(msg.getRequestId());

    } else if (msg.isCalcTask()) {
        gui.calculateServerTask(msg);

    } else if (msg.isFinalResult()) {
        gui.receiveFinalResult(msg.getRequestId(), msg.getRes());

    } else if (msg.isError()) {
        gui.showNetworkError("Сервер вернул ошибку.");

    } else {
        System.out.println("unknown message type: " + msg.getMsgType());
    }
}

```


} }

Приложение 2 – исходный код приложения-сервера

Приложение 2.1 – содержимое файла «CalcRequest.java»

```
package com.purpl.server6;

import java.util.ArrayList;

// msgType;requestId;partNum;a;b;h;res
// REGISTER;0;0;0;0;0;0
// REGISTER_OK;0;0;0;0;0;0
// CALC_REQUEST;1001;0;0.000001;10;0.001;0
// CALC_TASK;1001;1;3.333333;6.666666;0.001;0
// CALC_RESULT;1001;1;0;0;0;0.12345
// FINAL_RESULT;1001;0;0;0;0;1.23456

public class CalcRequest {
    private final int requestId;
    private final User mainUser;
    private final ArrayList<User> calcUsers;
    private final double a;
    private final double b;
    private final double h;
    private final int expectedResults;

    private int receivedResults;
    private double totalResult;

    public CalcRequest(int requestId, User mainUser, ArrayList<User> calcUsers,
        double a, double b, double h) {
        this.requestId = requestId;
        this.mainUser = mainUser;
        this.calcUsers = calcUsers;
        this.a = a;
        this.b = b;
        this.h = h;
        this.expectedResults = calcUsers.size();
        this.receivedResults = 0;
        this.totalResult = 0;
    }

    public int getRequestId() {
        return requestId;
    }

    public User getMainUser() {
```

```

        return mainUser;
    }

    public ArrayList<User> getCalcUsers() {
        return calcUsers;
    }

    public double getA() {
        return a;
    }

    public double getB() {
        return b;
    }

    public double getH() {
        return h;
    }

    public int getExpectedResults() {
        return expectedResults;
    }

    public int getReceivedResults() {
        return receivedResults;
    }

    public double getTotalResult() {
        return totalResult;
    }

    public void addResult(double result) {
        totalResult += result;
        receivedResults++;
    }

    public boolean isCompleted() {
        return receivedResults >= expectedResults;
    }

    @Override
    public String toString() {
        return "requestId=" + requestId
            + ", mainUser=" + mainUser
            + ", calcUsers=" + calcUsers.size()
            + ", a=" + a
            + ", b=" + b
            + ", h=" + h
            + ", received=" + receivedResults + "/" + expectedResults
            + ", totalResult=" + totalResult;
    }

```

} }

Приложение 2.2 – содержимое файла «Message.java»

```
package com.purpl.server6;

// msgType;requestId;partNum;a;b;h;res
// REGISTER;0;0;0;0;0;0
// REGISTER_OK;0;0;0;0;0;0
// CALC_REQUEST;1001;0;0.000001;10;0.001;0
// CALC_TASK;1001;1;3.333333;6.666666;0.001;0
// CALC_RESULT;1001;1;0;0;0;0.12345
// FINAL_RESULT;1001;0;0;0;0;1.23456

public class Message {
    public static final String TYPE_REGISTER = "REGISTER"; //enum
    public static final String TYPE_REGISTER_OK = "REGISTER_OK";
    public static final String TYPE_CALC_REQUEST = "CALC_REQUEST";
    public static final String TYPE_CALC_TASK = "CALC_TASK";
    public static final String TYPE_CALC_RESULT = "CALC_RESULT";
    public static final String TYPE_FINAL_RESULT = "FINAL_RESULT";
    public static final String TYPE_ERROR = "ERROR";

    public static final String TYPE_STATUS_CHECK = "STATUS_CHECK";
    public static final String TYPE_STATUS_OK = "STATUS_OK";

    private final String rawText;
    private final String msgType;
    private final int requestId;
    private final int partNum;
    private final double a;
    private final double b;
    private final double h;
    private final double res;

    public Message(String rawText) {
        this.rawText = rawText.trim();

        String[] parts = this.rawText.split(";");

        if (parts.length < 7) {
            throw new IllegalArgumentException("wrong message format: " + rawText);
        }

        this.msgType = parts[0].trim();
        this.requestId = Integer.parseInt(parts[1].trim());
        this.partNum = Integer.parseInt(parts[2].trim());
        this.a = Double.parseDouble(parts[3].trim());
        this.b = Double.parseDouble(parts[4].trim());
        this.h = Double.parseDouble(parts[5].trim());
    }
}
```

```

        this.res = Double.parseDouble(parts[6].trim());
    }

    public String getRawText() {
        return rawText;
    }

    public String getMsgType() {
        return msgType;
    }

    public int getRequestId() {
        return requestId;
    }

    public int getPartNum() {
        return partNum;
    }

    public double getA() {
        return a;
    }

    public double getB() {
        return b;
    }

    public double getH() {
        return h;
    }

    public double getRes() {
        return res;
    }

    public boolean isRegister() {
        return TYPE_REGISTER.equals(msgType);
    }

    public boolean isCalcRequest() {
        return TYPE_CALC_REQUEST.equals(msgType);
    }

    public boolean isCalcResult() {
        return TYPE_CALC_RESULT.equals(msgType);
    }

    public static String createRegisterOk() {
        return TYPE_REGISTER_OK + ";0;0;0;0;0;0";
    }

```

```

    public static String createError() {
        return TYPE_ERROR + ";0;0;0;0;0;0";
    }

    public boolean isStatusOk() {
        return TYPE_STATUS_OK.equals(msgType);
    }

    public static String createStatusCheck(int requestId) {
        return TYPE_STATUS_CHECK + ";"
            + requestId + ";"
            + 0 + ";"
            + 0 + ";"
            + 0 + ";"
            + 0 + ";"
            + 0;
    }

    public static String createCalcTask(int requestId, int partNum, double a, double b,
double h) {
        return TYPE_CALC_TASK + ";"
            + requestId + ";"
            + partNum + ";"
            + a + ";"
            + b + ";"
            + h + ";"
            + 0;
    }

    public static String createFinalResult(int requestId, double result) {
        return TYPE_FINAL_RESULT + ";"
            + requestId + ";"
            + 0 + ";"
            + 0 + ";"
            + 0 + ";"
            + 0 + ";"
            + result;
    }

    @Override
    public String toString() {
        return rawText;
    }
}

```

Приложение 2.3 – содержимое файла «Recipient.java»

```
package com.purpl.server6;

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.nio.charset.StandardCharsets;

// msgType;requestId;partNum;a;b;h;res
// REGISTER;0;0;0;0;0;0
// REGISTER_OK;0;0;0;0;0;0
// CALC_REQUEST;1001;0;0.000001;10;0.001;0
// CALC_TASK;1001;1;3.333333;6.666666;0.001;0
// CALC_RESULT;1001;1;0;0;0;0.12345
// FINAL_RESULT;1001;0;0;0;0;1.23456

public class Recipient {
    private static final int BUFFER_SIZE = 1024;

    private Recipient() {

    }

    public static DatagramPacket receive(DatagramSocket socket) throws Exception {
        byte[] buffer = new byte[BUFFER_SIZE];
        DatagramPacket packet = new DatagramPacket(buffer, buffer.length);

        socket.receive(packet);

        return packet;
    }

    public static String getText(DatagramPacket packet) {
        return new String(
            packet.getData(),
            0,
            packet.getLength(),
            StandardCharsets.UTF_8
        );
    }
}
```


Приложение 2.4 – содержимое файла «Register.java»

```
package com.purpl.server6;

import java.net.DatagramPacket;
import java.net.InetAddress;
import java.util.ArrayList;

// msgType;requestId;partNum;a;b;h;res
// REGISTER;0;0;0;0;0;0
// REGISTER_OK;0;0;0;0;0;0
// CALC_REQUEST;1001;0;0.000001;10;0.001;0
// CALC_TASK;1001;1;3.333333;6.666666;0.001;0
// CALC_RESULT;1001;1;0;0;0;0.12345
// FINAL_RESULT;1001;0;0;0;0;1.23456

public class Register {
    private static final ArrayList<User> users = new ArrayList<>();

    private Register() {

    }

    public static User registration(DatagramPacket packet, Message msg) {
        if (!Message.TYPE_REGISTER.equals(msg.getMsgType())) {
            System.out.println("this is not registration message.\n");
            return null;
        }

        InetAddress ip = packet.getAddress();
        int port = packet.getPort();

        User existingUser = findByAddress(ip, port);

        if (existingUser != null) {
            System.out.println("client already registered: " + existingUser + "\n");
            return existingUser;
        }

        String clientName = "Client-" + (users.size() + 1);

        User newUser = new User(ip, port, clientName);
        users.add(newUser);

        System.out.println("client registered: " + newUser + "\n");

        return newUser;
    }
}
```

```

    public static User findByAddress(InetAddress ip, int port) {
        for (User user : users) {
            if (user.getClientIp().equals(ip) && user.getClientPort() == port) {
                return user;
            }
        }

        return null;
    }

    public static ArrayList<User> getUsers() {
        return users;
    }

    public static ArrayList<User> getUsersCopy() {
        return new ArrayList<>(users);
    }

    public static int getUserCount() {
        return users.size();
    }

    public static void removeUser(User userToRemove) {
        users.remove(userToRemove);
        System.out.println("client removed: " + userToRemove + "\n");
    }

    public static void removeUsers(ArrayList<User> usersToRemove) {
        for (User user : usersToRemove) {
            removeUser(user);
        }
    }

    public static void printUsers() {
        System.out.println("=== registered users ===");

        if (users.isEmpty()) {
            System.out.println("no users registered\n");
            return;
        }

        for (User user : users) {
            System.out.println(user);
        }

        System.out.println();
    }
}

```

Приложение 2.5 – содержимое файла «Sender.java»

```
package com.purpl.server6;

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.nio.charset.StandardCharsets;

// msgType;requestId;partNum;a;b;h;res
// REGISTER;0;0;0;0;0;0
// REGISTER_OK;0;0;0;0;0;0
// CALC_REQUEST;1001;0;0.000001;10;0.001;0
// CALC_TASK;1001;1;3.333333;6.666666;0.001;0
// CALC_RESULT;1001;1;0;0;0;0.12345
// FINAL_RESULT;1001;0;0;0;0;1.23456

public class Sender {
    private Sender() {

    }

    public static void send(DatagramSocket socket, User user, String message) throws
Exception {
        byte[] data = message.getBytes(StandardCharsets.UTF_8);

        DatagramPacket packet = new DatagramPacket(
            data,
            data.length,
            user.getClientIp(),
            user.getClientPort()
        );

        socket.send(packet);

        System.out.println("sent to " + user + ": " + message);
    }
}
```

Приложение 2.6 – содержимое файла «ServerMain.java»

```
package com.purpl.server6;

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.HashMap;
import java.net.SocketTimeoutException;

// msgType;requestId;partNum;a;b;h;res
// REGISTER;0;0;0;0;0;0
// REGISTER_OK;0;0;0;0;0;0
// CALC_REQUEST;1001;0;0.000001;10;0.001;0
// CALC_TASK;1001;1;3.333333;6.666666;0.001;0
// CALC_RESULT;1001;1;0;0;0;0.12345
// FINAL_RESULT;1001;0;0;0;0;1.23456

public class ServerMain {
    private static final int SERVER_PORT = 5000;
    private static final int STATUS_CHECK_TIMEOUT_MS = 1500;

    private static final HashMap<Integer, CalcRequest> requests = new HashMap<>();

    public static void main(String[] args) {
        try (DatagramSocket socket = new DatagramSocket(SERVER_PORT)) {
            System.out.println("UDP server started on port " + SERVER_PORT);
            System.out.println("waiting for messages...\n");

            while (true) {
                DatagramPacket packet = Recipient.receive(socket);
                String text = Recipient.getText(packet);

                System.out.println("received from "
                    + packet.getAddress().getHostAddress()
                    + ":"
                    + packet.getPort()
                    + ": "
                    + text);

                try {
                    Message msg = new Message(text);

                    if (msg.isRegister()) {
                        handleRegister(socket, packet, msg);
                    } else if (msg.isCalcRequest()) {
```

```

        handleCalcRequest(socket, packet, msg);
    } else if (msg.isCalcResult()) {
        handleCalcResult(socket, msg);
    } else {
        System.out.println("unknown message type: " + msg.getMsgType() +
"\n");
    }

    } catch (Exception ex) {
        System.out.println("message processing error: " + ex.getMessage() +
"\n");
    }
}

} catch (Exception ex) {
    ex.printStackTrace();
}

}

private static void handleRegister(DatagramSocket socket, DatagramPacket packet,
Message msg) throws Exception {
    User user = Register.registration(packet, msg);

    if (user != null) {
        Sender.send(socket, user, Message.createRegisterOk());
        Register.printUsers();
    }
}

private static void handleCalcRequest(DatagramSocket socket, DatagramPacket packet,
Message msg) throws Exception {
    User mainUser = Register.findByAddress(packet.getAddress(), packet.getPort());

    if (mainUser == null) {
        System.out.println("calc request from unregistered user\n");
        return;
    }

    ArrayList<User> registeredUsers = Register.getUsersCopy();

    if (registeredUsers.isEmpty()) {
        System.out.println("no registered users for calculation\n");
        Sender.send(socket, mainUser, Message.createError());
        return;
    }

    int requestId = msg.getRequestId();

    if (requests.containsKey(requestId)) {

```

```

        System.out.println("request with this id already exists: " + requestId +
"\n");
        return;
    }

    System.out.println("checking active clients before calculation...\n");

    ArrayList<User> calcUsers = checkActiveUsers(socket, requestId,
registeredUsers);

    if (calcUsers.isEmpty()) {
        System.out.println("no active users for calculation\n");
        Sender.send(socket, mainUser, Message.createError());
        return;
    }

    CalcRequest request = new CalcRequest(
        requestId,
        mainUser,
        calcUsers,
        msg.getA(),
        msg.getB(),
        msg.getH()
    );

    requests.put(requestId, request);

    System.out.println("new calc request: " + request);
    System.out.println("distributing task between " + calcUsers.size() + " active
clients...\n");

    distributeTask(socket, request);
}

private static void distributeTask(DatagramSocket socket, CalcRequest request)
throws Exception {
    ArrayList<User> users = request.getCalcUsers();

    double a = request.getA();
    double b = request.getB();
    double h = request.getH();

    int clientCount = users.size();

    double length = b - a;
    double partLength = length / clientCount;

    for (int i = 0; i < clientCount; i++) {
        double partA = a + partLength * i;
        double partB;

```

```

        if (i == clientCount - 1) {
            partB = b;
        } else {
            partB = partA + partLength;
        }

        String taskMessage = Message.createCalcTask(
            request.getRequestId(),
            i,
            partA,
            partB,
            h
        );

        Sender.send(socket, users.get(i), taskMessage);
    }

    System.out.println();
}

private static void handleCalcResult(DatagramSocket socket, Message msg) throws
Exception {
    CalcRequest request = requests.get(msg.getRequestId());

    if (request == null) {
        System.out.println("result for unknown request: " + msg.getRequestId() +
"\n");
        return;
    }

    request.addResult(msg.getRes());

    System.out.println("received result for request "
        + msg.getRequestId()
        + ", part "
        + msg.getPartNum()
        + ": "
        + msg.getRes());

    System.out.println("progress: "
        + request.getReceivedResults()
        + "/"
        + request.getExpectedResults()
        + "\n");

    if (request.isCompleted()) {
        double totalResult = request.getTotalResult();

        String finalMessage = Message.createFinalResult(

```

```

        request.getRequestId(),
        totalResult
    );

    Sender.send(socket, request.getMainUser(), finalMessage);

    System.out.println("request completed: " + request.getRequestId());
    System.out.println("final result: " + totalResult);
    System.out.println("sent final result to: " + request.getMainUser() + "\n");

    requests.remove(request.getRequestId());
}
}

private static ArrayList<User> checkActiveUsers(DatagramSocket socket, int
requestId, ArrayList<User> users) throws Exception {
    ArrayList<User> activeUsers = new ArrayList<>();
    ArrayList<User> inactiveUsers = new ArrayList<>();

    String statusCheckMessage = Message.createStatusCheck(requestId);

    for (User user : users) {
        Sender.send(socket, user, statusCheckMessage);
    }

    int oldTimeout = socket.getSoTimeout();
    socket.setSoTimeout(STATUS_CHECK_TIMEOUT_MS);

    long startTime = System.currentTimeMillis();

    try {
        while (activeUsers.size() < users.size()) {
            try {
                DatagramPacket packet = Recipient.receive(socket);
                String text = Recipient.getText(packet);

                System.out.println("status response received from "
                    + packet.getAddress().getHostAddress()
                    + ":"
                    + packet.getPort()
                    + ": "
                    + text);

                Message msg = new Message(text);

                if (!msg.isStatusOk()) {
                    System.out.println("not status message during status check: " +
msg.getMsgType());
                    continue;
                }
            }

```



```

        if (msg.getRequestId() != requestId) {
            System.out.println("status ok for another request: " +
msg.getRequestId());
            continue;
        }

        User user = Register.findByAddress(packet.getAddress(),
packet.getPort());

        if (user == null) {
            System.out.println("STATUS_OK from unknown user\n");
            continue;
        }

        if (!activeUsers.contains(user)) {
            activeUsers.add(user);
            System.out.println("client is active: " + user);
        }

        long elapsed = System.currentTimeMillis() - startTime;

        if (elapsed >= STATUS_CHECK_TIMEOUT_MS) {
            break;
        }

    } catch (SocketTimeoutException ex) {
        break;
    }
}
} finally {
    socket.setSoTimeout(oldTimeout);
}

for (User user : users) {
    if (!activeUsers.contains(user)) {
        inactiveUsers.add(user);
    }
}

if (!inactiveUsers.isEmpty()) {
    System.out.println("\ninactive clients:");

    for (User user : inactiveUsers) {
        System.out.println(user);
    }

    System.out.println();
}

```

```
        System.out.println("active clients: " + activeUsers.size() + "/" + users.size()
+ "\n");

        Register.removeUsers(inactiveUsers);

        return activeUsers;
    }
}
```

Приложение 2.7 – содержимое файла «User.java»

```
package com.purpl.server6;

import java.net.InetAddress;

// msgType;requestId;partNum;a;b;h;res
// REGISTER;0;0;0;0;0;0
// REGISTER_OK;0;0;0;0;0;0
// CALC_REQUEST;1001;0;0.000001;10;0.001;0
// CALC_TASK;1001;1;3.333333;6.666666;0.001;0
// CALC_RESULT;1001;1;0;0;0;0.12345
// FINAL_RESULT;1001;0;0;0;0;1.23456

public class User {
    private final InetAddress clientIp;
    private final int clientPort;
    private final String clientName;

    public User(InetAddress clientIp, int clientPort, String clientName) {
        this.clientIp = clientIp;
        this.clientPort = clientPort;
        this.clientName = clientName;
    }

    public InetAddress getClientIp() {
        return clientIp;
    }

    public int getClientPort() {
        return clientPort;
    }

    public String getClientName() {
        return clientName;
    }

    @Override
    public String toString() {
        return clientName + " " + clientIp.getHostAddress() + ":" + clientPort;
    }
}
```